Автономная некоммерческая организация высшего образования
«Университет Иннополис»

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(БАКАЛАВРСКАЯ РАБОТА)
по направлению подготовки
09.03.01 «Информатика и вычислительная техника»

FINAL QUALIFICATION WORK
(BACHELOR'S THESIS)
Academic Program
09.03.01 «Computer Science»

Направленность (профиль) образовательной программы
«Анализ данных и искусственный интеллект»

Field of Study:
«Data Science and Artificial Intelligence»

Тема	**Контекстный кросс-энкодер для высокоточного текстового реранжирования при поиске информации на русском языке**
Topic	**A Contextual Cross-Encoder for High-Precision Textual Reranking in Russian Language Information Retrieval**

Работу выполнил / *Prepared by*	**Дыдалин Дмитрий Сергеевич / Dydalin Dmitrii Sergeevich**	подпись / signature
Руководитель выпускной квалификационной работы / *Final Qualification Work Supervisor*	**Лукманов Рустам Абубакирович / Lukmanov Rustam Abubakirovich**	подпись / signature

Иннополис, Innopolis, 2026

Contents

1 Introduction **13**

1.1 Definition of the task 13

1.1.1 The Importance of High-Precision Textual Ranking in Modern NLP 13

1.1.2 The Two-Stage IR Pipeline: Retrieval and Reranking . . . 14

1.1.3 The Critical Role of Reranking in Retrieval-Augmented Generation 16

1.1.4 Motivation and The Specialization Gap 17

1.1.5 Research Questions 17

1.1.6 Experimental approach 18

2 Literature Review **20**

2.1 Classical IR as First-Stage Retrieval 20

2.1.1 TF-IDF and BM25 20

2.1.2 Role in Reranking Pipelines 21

2.1.3 The Limits of Lexical Matching and the Path to Semantic Retrieval 22

2.2 Deep Learning Methods of Textual Retrieval 22

2.2.1 Taxonomy of Neural Retrieval Models 23

2.2.2 Bi-Encoder Architectures 25
2.2.3 Cross-Encoder Architectures 31
2.3 Datasets and the Russian Training-Data Gap 37
2.3.1 Methodological Role of Training Data 37
2.3.2 Russian Evaluation Resources and Training-Data Fragmentation 38
2.4 Weak Supervision, Noise, and Consistency Filtering 39
2.5 Hard Negative Mining and False-Negative Risk 41
2.6 Contrastive Training and Effective Batch Size 42
2.7 Cross-Lingual Augmentation and Distillation Alternatives 43
2.8 Evaluation Integrity and Benchmark Contamination 44
2.9 Open Challenges and Positioning 45
2.9.1 Data Quality and Russian-Specific Supervision 45
2.9.2 Compact Cross-Encoders and Inference Cost 45
2.9.3 Synthesis: Positioning of This Thesis 46

3 Methodology 47
3.1 General Architecture of the Proposed Pipeline 47
3.2 Data Processing 49
3.2.1 Unification and Preparation of the Source Data 49
3.2.2 Formation of Training and Validation Splits 50
3.2.3 Filtering Training Pairs by the Consistency Criterion . . . 51
3.2.4 Hard Negative Mining 53
3.2.5 Cross-Lingual Data Augmentation and Final Dataset Compilation 55
3.3 Training Procedure 56
3.3.1 Architecture Selection Criteria 56

3.3.2 Batching and Sampling Strategy 57
3.3.3 Loss Function and Negative Composition 58
3.3.4 Training Validation Metrics 60
3.4 Evaluation Protocol 62
3.4.1 Data Isolation and Anti-Contamination Measures 62
3.4.2 Two-Stage Evaluation Pipeline 62
3.4.3 Benchmark Coverage and Aggregation 63
3.4.4 Benchmark Evaluation Metrics 64
3.4.5 Ablation Study Design 64
3.5 Methodology Summary 66

4 Experiments **67**
4.1 Software and Computational Environment 67
4.2 Training Data 68
4.2.1 Source Datasets 69
4.2.2 Consistency Filtering 70
4.2.3 Hard Negative Mining 72
4.2.4 Final Corpus Composition 73
4.3 Training Configuration 77
4.3.1 Model and Hyperparameters 77
4.4 Evaluation Protocol 79
4.4.1 Dataset Selection 79
4.4.2 Contamination-Control Protocol 80
4.4.3 Evaluation Mixture 80
4.4.4 First-Stage Retrieval Setup 82
4.4.5 Evaluation Metrics 82
4.4.6 Summary 83

4.5 Ablation: Consistency Filtering 83
4.5.1 Ablation Design 83
4.5.2 Results 84
4.5.3 Analysis 86
4.6 Ablation: Hard Negative Sampling 87
4.6.1 Ablation Design 87
4.6.2 Results 88
4.6.3 Analysis 90
4.7 Ablation: SLERP 92
4.7.1 Ablation Design 92
4.7.2 Results 93
4.7.3 Analysis 95

5 Evaluation and Discussion 97
5.1 Quantitative Evaluation 97
5.1.1 Experimental Setup 97
5.1.2 Results 98
5.1.3 Analysis 99
5.2 Qualitative Error Analysis 103
5.2.1 Sampling Procedure 103
5.2.2 Error Taxonomy 104
5.2.3 Discussion 107
5.3 Research Objectives Assessment 109
5.3.1 O1: Architecture Selection and Comparative Effectiveness 109
5.3.2 O2: Heterogeneous Training Corpus 109
5.3.3 O3: Ablation of Pipeline Optimisations 110
5.4 Limitations and Future Work 111

5.4.1 Fixed Choice of the Retrieval Model 111
5.4.2 Absence of Dataset Sampling and Loss Weighting Strategies 112
5.4.3 SLERP Across Different-Seed Training Runs 112
5.4.4 Absence of Optimiser Hyperparameter Tuning 113
5.4.5 Limited Comparison of Cross-Encoder Base Models . . . 113

References **115**

Statement of Individual Contributions **127**

A RusBEIR Evaluation Datasets: Contamination Risk Analysis **130**

B Per-Dataset Evaluation Results **133**

List of Tables

2.1 Architectural Trade-Offs in Neural Textual Retrieval 26

2.2 Comparison of Bi-Encoder Architectures: Main Characteristics and MTEB Performance 31

2.3 Examples of Reranking and Retrieval Resources Used in High-Resource Settings 38

2.4 Russian Retrieval Resources and Their Training Challenges 40

4.1 Source datasets used in the construction of the final training corpus. 69

4.2 Training corpus size under three consistency-filtering strategies. V1: no filtering; V2: k=30; V3: k=5. 71

4.3 Query dropout due to insufficient hard negatives (N=15), training partition. V1: unfiltered; V2: k=30; V3: k=5. habr-qna has no V1 log. 73

4.4 Query dropout due to insufficient hard negatives (N=15), validation partition (mined in unfiltered form). 74

4.5 Final corpus sizes after the assembly script (N=8 hard negatives per row, cross-lingual permutations applied). V1: unfiltered; V2: k=30; V3: k=5. The validation set is built from the unfiltered cleaned corpus (single variant). 75

4.6 RusBEIR evaluation datasets. The nine active datasets (weight $= 1/9$) form the primary evaluation mixture; two are excluded due to direct passage-corpus overlap with training sources. Full per-dataset contamination analysis is in Appendix A. 82

4.7 Impact of consistency filtering on benchmark quality. Best checkpoint per run (by NDCG@10), aggregated over the nine active RusBEIR datasets. bge-reranker-v2-m3 is an off-the-shelf reference, not part of the ablation. Bold marks the best value among the three variants per column. 86

4.8 SLERP across the best checkpoints of the three consistency-filtering runs (V1, V2, V3). Mean across the eleven RusBEIR datasets. Bold marks the best value per column. 94

4.9 SLERP across five consecutive checkpoints of the V2 (k=30) training run. Mean across the eleven RusBEIR datasets. Bold marks the best value per column. 95

5.1 Comparison of retrieval models on the rusBEIR benchmark (macro-average over 9 datasets; ruscibench-retrieval_dev and ruscidocs_test excluded due to English queries). The best score per column is **bolded**. Params denotes model size in millions of parameters. All scores are from a single training run; no averaging over multiple seeds was performed. 98

5.2 Cross-encoder error taxonomy. Frequencies are counts and percentages over the 132-sample analysis set (seed = 42, top-5 failure criterion, 8 rusBEIR datasets). 104

A.1 Contamination risk analysis for all eleven RusBEIR candidate datasets. *Direct*: passage corpus is the same as a training source. *Corpus*: passages drawn from the same collection with independent queries/annotations. *Domain*: related domain but different corpus. *None*: no identified overlap. 132

B.1 Per-dataset NDCG@10. Best per row in **bold**. 133

B.2 Per-dataset MAP@10. Best per row in **bold**. 134

B.3 Per-dataset P@3. Best per row in **bold**. 134

B.4 Per-dataset MRR@10. Best per row in **bold**. 135

List of Figures

4.1 Passage token-length distribution by dataset compared with the global passage distribution. Lengths are computed with the RuModernBERT-base tokenizer. The figure reports passage lengths only, because query lengths are substantially more homogeneous across datasets. 70

4.2 Thematic distribution of the V3 (k=5) filtered training corpus by source category (6,279,747 total pairs). 76

4.3 Contamination-aware evaluation pipeline for constructing the final benchmark. 81

4.4 Benchmark metrics versus training step for the three consistency-filtering variants, aggregated over the nine active RusBEIR datasets (equal weight). The dashed line marks bge-reranker-v2-m3. Checkpoints before step 1,000 are omitted. 85

4.5 Benchmark metrics versus training step for the three hard-negative sampling conditions (0HN+2RN, 8HN+2RN, 15HN+2RN), aggregated over the nine active rusBEIR datasets (equal weight). All three runs share the V2 (k=30) training corpus and the shared hyperparameters of §4.3.1. 89

4.6 Experiment 1: best checkpoints of the V1, V2, V3 runs versus their SLERP merge. Mean over the eleven RusBEIR datasets. 94

4.7 Experiment 2: SLERP across five consecutive checkpoints from the V2 (k=30) run versus the merged model. Mean over the eleven RusBEIR datasets. 95

5.1 Macro-averaged retrieval metrics over the nine active rusBEIR datasets for all evaluated models. Each group of bars corresponds to one metric; bar colour identifies the model (see legend). Exact values are reported in Table 5.1; per-dataset breakdowns are in Appendix B. 99

5.2 NDCG@10 on the rusBEIR macro-average (9 datasets, out-of-distribution) across all evaluation checkpoints during fine-tuning of RuModernBERT-base (8HN+2RN). Dashed horizontal lines mark the baseline scores for BM25, Qwen3-Reranker-0.6B, and bge-reranker-v2-m3; exact values are in Table 5.1. 101

5.3 In-training dynamics of the 8HN+2RN fine-tuning run of RuModernBERT-base: training loss (top), gradient norm before clipping (middle), and learning rate (bottom) versus optimisation step. 102

Abstract

We train a Russian cross-encoder reranker on top of RuModernBERT-base (150 M parameters) using a heterogeneous corpus assembled from eleven Russian sources and selected English domain-specific data. The pipeline combines consistency-based filtering of weakly supervised pairs against a dense retriever, mixed hard- and random-negative sampling with margin-based mining, and dataset-aware mini-batching to limit padding overhead; spherical linear interpolation of checkpoints is evaluated as a low-cost post-training step.

On a contamination-controlled nine-dataset rusBEIR macro-average, the model reaches NDCG@10 of 0.560, MAP@10 of 0.432, MRR@10 of 0.597 and P@3 of 0.311, outperforming Qwen3-Reranker-0.6B on every reported metric at roughly one quarter of its parameters, and trailing bge-reranker-v2-m3 (568 M) by 2.7 pp NDCG@10.

Chapter 1

Introduction

1.1 Definition of the task

1.1.1 The Importance of High-Precision Textual Ranking in Modern NLP

Information Retrieval (IR) is the task of locating relevant data resources, whether documents, passages, or entities, from a large corpus in response to a user query [1]. For decades, IR systems have relied on classical lexical matching techniques. With the advent of deep learning, the paradigm of textual search has shifted toward semantic methods, driven by the increasing complexity of user information needs. Traditional methods struggle to capture deep semantic relationships and often fail to match documents containing relevant concepts expressed through synonyms or paraphrasing [2].

The transition to deep semantic matching, enabled by pretrained language models such as BERT, is now central to building robust and high-performance systems [3], [4]. Accurate semantic ranking underpins knowledge-intensive ap-

plications in production, including web search, recommendation engines, and question-answering systems used in domains such as medical research and legal discovery [2].

The growing deployment of AI assistants has increased the demand for high-precision textual ranking, since retrieval quality directly affects the factual reliability of generated outputs [2]. Retrieval components are the primary mechanism for grounding model outputs in verifiable external evidence, and by supplying such evidence they reduce the tendency of language models to fabricate details, a phenomenon commonly termed hallucination [5]. The quality of the IR component, and in particular the final ranking step, therefore has a direct effect on the reliability of the surrounding Natural Language Processing (NLP) application. Failure to rank the most relevant documents at the top undermines the function of these AI systems, which is why ranking techniques remain an active area of research.

1.1.2 The Two-Stage IR Pipeline: Retrieval and Reranking

In large-scale information systems, ranking the entire document corpus with high precision is computationally impractical under realistic latency constraints. The standard solution is a two-stage retrieval pipeline that trades some accuracy for tractable computation [6]. The pipeline divides the task into a retrieval phase and a reranking phase, each using models optimized for different constraints.

- Stage 1: Textual Retrieval The primary objective of the initial retrieval stage is to efficiently generate an initial high-recall candidate set from the entire document corpus, which may include billions of documents. This stage typically employs computationally faster models, such as bi-encoders or established sparse lexical models. These models determine semantic or lexical

similarity using low-cost and high-speed operations, most commonly the dot product or cosine similarity between precomputed vector representations. Because document embeddings are calculated offline, the retrieval process primarily involves a rapid approximate nearest-neighbor search on a large index. At this stage, the emphasis is on speed and recall, ensuring that all potentially relevant documents are captured, even if they are not perfectly ordered.

- Stage 2: Textual Reranking The reranking stage is dedicated to refining the rank order of the limited candidate set provided in Stage 1. The goal is to maximize precision, ensuring that the most relevant documents appear at the top of the resultant list. This phase utilizes significantly more complex models than the previous phases. Historically, reranking has relied on methods that involve manually defined features and learning-to-rank losses. Since the widespread adoption of transformer architecture, cross-encoder (CE) models, such as those based on BERT, have become the dominant approach to reranking [7].

CEs achieve superior relevance scoring by processing concatenated queries and document segments. This allows cross-attention between each token in the query and document. These token-level interactions let the model capture fine-grained semantic dependencies and produce more accurate relevance scores than bi-encoders [8]. However, this detailed token interaction incurs a high computational cost owing to its quadratic complexity in the sequence length ($O(L_q \cdot L_d)$). This constraint limits the number of documents that can be processed in real time and is why an efficient retrieval stage is needed. The distinction between retrieval and reranking lies in this difference in interaction: retrieval performs shallow

pre-indexed comparisons for speed, whereas reranking sacrifices speed for deep dynamic interaction to maximize accuracy.

1.1.3 The Critical Role of Reranking in Retrieval-Augmented Generation

The rise of Retrieval-Augmented Generation (RAG) has made reranking even more important. Introduced in [5], the RAG framework integrates a nonparametric retrieval mechanism with a parametric sequence-to-sequence generator. It combines static knowledge stored in model parameters (parametric memory) with real-time access to external data (nonparametric memory).

The effectiveness of an RAG system largely depends on the quality of the context passages it retrieves. When the passages provided to the Large Language Model (LLM) are irrelevant or contain noise, the quality of the LLM output degrades. Reranking is a critical mechanism for optimizing passage quality: it improves the rank order of relevant documents, often with much higher accuracy than single-stage retrievers, and is therefore essential for the factual accuracy of RAG outputs [1], [2].

The CE reranker affects RAG accuracy in two ways. First, reranking is essential when the number of retrieved passages exceeds the LLM's input context window. By promoting the most relevant passages, reranking ensures that the limited context budget contains the highest concentration of factual evidence and keeps noisy chunks out of generation. Second, even when the context window can accommodate all retrieved passages, the quality of the final generation is sensitive to their relative position. LLMs often struggle to use information located in the middle of a long context, a behavior known as the "Lost in the Middle" effect

[9]. Reranking is therefore needed both to filter passages and to position the most relevant information at the beginning or end of the context, where LLMs attend more effectively.

In short, RAG has shifted reranking from a secondary search optimization to a primary determinant of LLM output reliability. Progress in retrieval-augmented generation therefore depends on advances in the reranking layer.

1.1.4 Motivation and The Specialization Gap

The field of Russian IR has access to evaluation frameworks such as RusBEIR [10] and multilingual training corpora including Russian-language retrieval subsets of large-scale benchmarks [11]. Nevertheless, high-quality Russian-specific reranking annotations remain substantially scarcer than their English counterparts. State-of-the-art multilingual CEs that span from 70 to over 100 languages involve cross-lingual trade-offs: optimizing a single model across diverse language families reduces the relevance-scoring precision attainable for any individual language [12].

This study addresses this specialization gap by applying dedicated monolingual training, previously demonstrated for Russian Transformer LMs [13], to the more accurate CE architecture, combined with modern architectural optimizations and a heterogeneous training corpus, in order to improve Russian retrieval precision.

1.1.5 Research Questions

RQ1: Can a specialized Russian CE reranker, trained on a heterogeneous corpus of Russian retrieval data and English domain-specific data with targeted negative sampling strategies, surpass existing general-purpose multilingual CE competitors in ranking effectiveness on Russian retrieval benchmarks, as measured

by NDCG@10 and MRR?

RQ2: To what extent do specific training pipeline optimizations—consistency-based filtering of weakly supervised training data and mixed hard/random negative sampling—individually contribute to the reranking precision of the specialized CE, as measured on held-out Russian retrieval benchmarks?

1.1.6 Experimental approach

The project was structured with three core objectives.

- O1: To select and adapt a modern, computationally efficient Russian Transformer architecture for CE operation, prioritizing an extended context window (up to 8,192 tokens) and deployment-practical inference cost, while ensuring robust foundational Russian linguistic capabilities. This choice additionally enables comparative analysis of ranking effectiveness relative to multilingual CEs that are substantially larger in parameter count.

- O2: To construct a heterogeneous training corpus combining Russian-language general retrieval data with English domain-specific retrieval data (NQ, SWIM-IR) [14], [15], hypothesizing that exposure to English domain-specific supervision will improve the model's robustness on Russian texts containing significant English vocabulary—technical terms, identifiers, and labels common in scientific and technical queries—thereby improving retrieval precision in these domains without requiring Russian-native domain-specific annotations.

- O3: To empirically quantify, through controlled ablation studies, the marginal contribution of two training pipeline decisions to reranking precision: (a) the

performance gain attributable to consistency-based filtering of weakly supervised Russian training data over unfiltered training, and (b) the comparative effectiveness of mixed hard/random negative sampling against both pure-random and pure-hard-negative baselines—establishing whether the added complexity of these techniques yields practically significant improvements on held-out Russian retrieval benchmarks.

Chapter 2

Literature Review

2.1 Classical IR as First-Stage Retrieval

Classical IR matters here primarily as the first stage of modern two-stage search pipelines. The Probability Ranking Principle (PRP) states that documents should be ranked by their estimated probability of relevance to a user query [16]. Neural rerankers estimate that probability differently, but the pipeline objective stays the same: a fast retriever produces a high-recall candidate set, and a more expensive cross-encoder refines the ordering.

2.1.1 TF-IDF and BM25

Term Frequency–Inverse Document Frequency (TF-IDF) introduced a simple and effective way to score documents using corpus statistics [17]. Its central idea is term specificity: terms that occur frequently in a document but rarely in the corpus receive higher weight. TF-IDF is computationally efficient and suitable for sparse candidate retrieval, but its linear treatment of term frequency and lack of document-length normalization limit ranking quality.

Okapi BM25 addresses these limitations while preserving the efficiency of lexical retrieval [18]. BM25 adds term-frequency saturation, controlled by k_1, so repeated occurrences of the same term provide diminishing returns, and document-length normalization, controlled by b, so long documents are not rewarded merely for containing more terms. These properties made BM25 a durable baseline and a common first-stage retriever in neural IR pipelines.

2.1.2 Role in Reranking Pipelines

In a two-stage architecture, TF-IDF and BM25 are not competitors to cross-encoders but complementary mechanisms. They cheaply identify candidate passages that satisfy lexical relevance constraints, after which neural rerankers can apply deeper semantic interaction to a much smaller candidate list. BM25 also provides a useful reference point for neural ranking: recent analyses suggest that transformer rerankers can implicitly recover frequency, saturation, and inverse-document-frequency-like signals [19].

Classical lexical retrieval is also important for training data construction. Hard negative mining relies on retrieving passages that appear plausible for a query but are not labeled as relevant. BM25 is frequently used for this purpose because it produces lexically close negatives, while dense retrievers produce semantically close negatives. The methodology of this thesis uses a dense retrieval model for filtering and mining, but the underlying logic follows the same first-stage retrieval principle: a cheaper model defines a candidate neighborhood, and the cross-encoder learns to discriminate among difficult candidates within that neighborhood.

2.1.3 The Limits of Lexical Matching and the Path to Semantic Retrieval

Lexical matching methods such as TF-IDF and BM25 assume that relevant documents share surface-form terms with the query. This assumption fails when relevant passages express the same concept through synonyms, paraphrases, or domain-specific reformulations. The vocabulary mismatch problem—where a relevant document uses entirely different words from the query—is a well-documented ceiling on purely lexical recall.

An early response came from distributional word embedding models such as Word2Vec and GloVe [20], [21]. These models represent words as dense vectors learned from co-occurrence statistics, so words used in similar contexts end up geometrically close. Retrieval could then operate on semantic proximity rather than exact term matching.

Static embedding models, however, face a known architectural limitation: each word receives a single fixed vector regardless of context. The word "bank" gets the same representation in "river bank" and "investment bank." For reranking, where relevance is sensitive to how query meaning interacts with specific passage context, this context-independence limits discriminative power. Static embeddings were an important intermediate step—they showed that meaning could be encoded in continuous vector spaces—but they lack the context-sensitivity that cross-encoder reranking requires.

2.2 Deep Learning Methods of Textual Retrieval

The vocabulary mismatch ceiling of lexical methods and the lack of context in static embeddings both push toward contextual deep learning models. Deep

learning changed IR substantially, moving retrieval systems away from cheap but semantically limited approaches toward architectures that represent words in their full surrounding context [22].

BERT [3], introduced in 2018 via the transformer architecture [23], offered a bidirectional language model with strong contextual understanding and effective transfer learning. However, the direct application of BERT to large-scale retrieval tasks presents immediate scalability challenges. Standard BERT implementations, originally designed for sequence pair regression, require processing the entire corpus with a full transformer stack for every query-document pair. For example, finding the pair with the highest similarity in 10,000 sentences using BERT [3] or Robustly Optimized BERT Pretraining Approach (RoBERTa) [24] requires approximately 65 h of computation [4]. Such latency makes vanilla BERT unusable as a deployed retrieval system, regardless of its ranking accuracy.

This bottleneck drove the development of efficient retrieval architectures. Sentence-BERT (SBERT) [4] reduced the same retrieval task to approximately 5 s, confirming that scalability and inference efficiency are core design metrics alongside ranking quality.

2.2.1 Taxonomy of Neural Retrieval Models

Neural retrieval architectures differ in how much interaction they allow between query Q and document D at scoring time. That interaction level determines the core trade-off between speed and accuracy, summarized in Table 2.1.

Bi-Encoder Models: Zero-Interaction Bi-encoders, exemplified by early SBERT [4] and modern models such as Multilingual E5 [25], encode query Q and document D entirely separately using two often identical, weight-tied neural networks. The

output is a fixed-size dense vector for each text. Relevance is then computed as a simple similarity metric, such as cosine similarity, between these two vectors [4].

Because all document embeddings can be precomputed and indexed offline, retrieval is a fast nearest-neighbor search over the vector space [4]. The downside is an information bottleneck: the fixed-size vector must compress all semantic content without any query-aware context, which limits accuracy compared to full-interaction models [26].

Cross-Encoder Models: Full-Interaction Cross-encoders are currently the most accurate architecture for relevance scoring in IR [27], [28]. They concatenate query and document into a single input sequence and process it through the full transformer stack. This full-interaction mechanism allows the model to compute contextualized relationships between query and document tokens at every layer of the network [27], producing higher ranking precision than any other neural retrieval approach [28].

This accuracy comes at a cost. CEs based on large transformer models are computationally expensive and can score only a small number of documents within a tight latency budget [28]. Because the interaction occurs during inference, a CE must be run for every candidate query-document pair [27], which makes it unsuitable for first-stage retrieval over large corpora.

Late Interaction Models: Multi-vector Late interaction models such as ColBERT sit between bi-encoders and cross-encoders [29], [30]. Like bi-encoders, they encode queries and documents separately, so document representations can be precomputed. Unlike dense retrievers that compress each document into a single vector, ColBERT retains an embedding for every token.

The scoring mechanism itself is the key difference. Rather than comparing

single summary vectors, ColBERT computes a fine-grained token-level similarity between query and document embeddings at the final stage. This approximates the query-document attention of a cross-encoder without the per-pair inference cost [29], recovering relevance signals that single-vector representations miss.

Late interaction models consistently outperform conventional bi-encoders on IR benchmarks [29], [31]. Token-level interactions give a finer-grained relevance signal than single-vector comparisons, which is what makes them effective for passage search.

Despite this, late interaction models are not used in this thesis as the reranking architecture. The principal reasons are practical. First, storing per-token embeddings for every passage in the corpus requires substantially more index memory than single-vector bi-encoder indices; at corpus scale, this overhead is significant. Second, ColBERT's tokenizer and vocabulary were designed for English, and multilingual or Russian-specific ColBERT variants are not yet established as mature alternatives. The thesis therefore adopts a standard CE architecture, which requires no persistent passage index and is directly supported by multilingual and Russian-focused pretrained encoders.

2.2.2 Bi-Encoder Architectures

BERT-based Retrieval Adaptation The Bidirectional Encoder Representations from Transformers model [3] established the viability of deep bidirectional context modeling, leveraging unsupervised pretraining objectives such as Masked Language Modeling (MLM) and Next Sentence Prediction. When adapted for dense retrieval, BERT serves as the base encoder for both the query and passage.

Adapting BERT for bi-encoder use faces a known limitation: vanilla BERT representations, when averaged or pooled to create sentence-level embeddings,

TABLE 2.1
Architectural Trade-Offs in Neural Textual Retrieval

Architecture Type	**Query-Document Interaction**	**Efficiency (Index Time)**	**Effectiveness (Accuracy)**	**Key Models**
Bi-Encoder (Dense)	Zero (Separate vector encoding)	Very Fast, Scalable	Lower (Information bottleneck)	Multilingual E5, XLM-RoBERTa [32], [33]
Cross-Encoder (Full)	Full attention across Q and D tokens	N/A (Interaction at inference)	Highest (Deep contextual fusion)	GTE Multilingual Reranker, Qwen3 Reranker [34], [35]
Late Interaction (Multi-vector)	Token-wise MaxSim over document embeddings	Fast (Separate encoding)	High (Approximates Cross-Encoder accuracy)	ColBERT [29]

perform poorly in semantic similarity tasks [4]. This comes from BERT's training objectives, which focus on MLM and next-sentence prediction rather than learning semantically meaningful sentence representations. Directly using BERT embeddings for similarity can produce counterintuitive results, where semantically unrelated sentences appear more similar than related ones [36].

Sentence-BERT SBERT [4] addressed BERT's limitations in sentence representation through a siamese network architecture. SBERT modifies the pretrained BERT model by adding pooling operations and contrastive learning objectives designed for sentence similarity tasks. The architecture uses twin BERT networks with shared parameters to encode sentence pairs independently, producing fixed-size sentence embeddings that can be compared efficiently.

SBERT employs different objective functions depending on the downstream task. For natural language inference classification tasks, the model concatenates sentence embeddings with their element-wise difference as $[\mathbf{u}; \mathbf{v}; |\mathbf{u} - \mathbf{v}|]$, which is then fed into a softmax classifier. This training step utilizes cross-entropy loss.

For semantic textual similarity regression tasks, SBERT uses the cosine similarity between embeddings $\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|\|\mathbf{v}\|}$ and optimizes the mean squared error loss. Training draws on datasets including Stanford Natural Language Inference [37], Multi-Genre Natural Language Inference [38], and the Semantic Textual Similarity Benchmark [39], supporting generalization across different semantic tasks.

RoBERTa and XLM-RoBERTa RoBERTa [24] refines BERT's training without changing the architecture. It drops the Next Sentence Prediction objective entirely and uses only MLM with dynamic masking: rather than masking tokens once during preprocessing, new mask patterns are generated for each training epoch. RoBERTa also uses byte-level Byte Pair Encoding (BPE) tokenization instead of BERT's WordPiece. The pretraining loss is standard cross-entropy applied to the prediction logits.

XLM-RoBERTa [33] extends this approach to a multilingual setting. It was pretrained with the MLM objective on a 2.5 TB corpus of CommonCrawl data spanning 100 languages. The resulting CC-100 dataset substantially increased the volume of multilingual training data available for downstream tasks.

E5 The E5 (embeddings from multimodal, multi-task, and multi-source information) model family [25], [32] is a significant step in bi-encoder design. Like SBERT [4], E5 uses a shared encoder architecture that produces independent, fixed-size vector representations, allowing efficient similarity computation through vector operations. Where SBERT relied on small, carefully curated supervised datasets, E5 shifted to massive weak supervision. The model distinguishes between query and passage roles through the prefix identifiers "query:" and "passage:" added to the respective input texts, enabling task-aware embedding generation.

E5 was trained with contrastive learning in both the initial weakly supervised

pretraining phase and the subsequent fine-tuning stage. The Colossal Clean Text Pairs dataset [25] was used in the first stage. Data collection mined semi-structured sources to extract text pairs (q, p), where q is a query and p is a passage of arbitrary length. During initial contrastive pretraining, E5 used in-batch negative sampling with large batch sizes. The model was then fine-tuned on a mix of high-quality labeled datasets to improve generalization across tasks. The contrastive learning framework uses the InfoNCE loss function:

$$\mathcal{L} = -\log \frac{\exp(\text{sim}(h_i, h_i^+)/\tau)}{\sum_{j=1}^{B} \exp(\text{sim}(h_i, h_j^+)/\tau)} \tag{1}$$

where $\text{sim}(\cdot, \cdot)$ denotes the similarity function, τ denotes the temperature parameter, and B denotes the batch size. The result is a universal embedding model that generalizes across retrieval and semantic similarity tasks without task-specific architectures or extensive manual annotation.

BGE M3 BGE M3-Embedding [40] is a highly adaptable multilingual text-embedding model constructed on a cross-lingual transformer framework, specifically XLM-RoBERTa-large. This model was enhanced to accommodate extended inputs, reaching 8,192 tokens through RetroMAE [41] pretraining. The training process followed a multistage pipeline. In the first stage, it underwent unsupervised contrastive learning with approximately 1.2 billion text pairs sourced from various multilingual corpora. The model was then fine-tuned on compiled labeled retrieval datasets and augmented with synthetic long-document QA examples generated by GPT-3.5 to improve long-document retrieval. During the training phase, M3-Embedding concurrently learns three retrieval objectives: standard dense retrieval utilizing $[CLS]$ token embedding for query–document similarity, sparse lexical retrieval employing learnable per-token term-weight projections, and ColBERT style

[29] multi-vector retrieval via late interaction on token embeddings. Each retrieval head was trained using a contrastive InfoNCE loss. The model also incorporates a self-knowledge distillation scheme in which the scores from all three retrieval modes are aggregated to form a joint "teacher" relevance score. Each head was trained to align with this ensemble output as a soft target along with its native target. The final training objective can be formalized as

$$\mathcal{L}_{final} = \lambda_1 \mathcal{L}_{retrieval} + \lambda_2 \mathcal{L}_{distillation} \tag{2}$$

where $\mathcal{L}_{retrieval}$ denotes the combined InfoNCE loss across the retrieval heads, $\mathcal{L}_{distillation}$ represents the self-knowledge distillation loss aligning each head with the ensemble teacher signal, and λ_1, λ_2 are scalar weights that control their relative contributions. This loss allows a single embedding space to perform effectively across languages, retrieval functions, and input granularities.

Qwen3 Embeddings The Qwen3 Embedding [35] series is based on the pretrained Qwen3 transformer [42] backbones. Qwen3 employs causal self-attention, in contrast to BERT, which utilizes bidirectional self-attention. To address this difference, the model appends a special $[EOS]$ token. The final embedding is obtained from the hidden state of the last transformer layer, corresponding to the position of the $[EOS]$ token. To enable instruction-aware embeddings, the model concatenates task-specific instructions with queries into a unified input context while processing documents independently. The training followed a multistage contrastive framework.

1. The model underwent large-scale weakly supervised pretraining on approximately 150 million synthetic text pairs. An extensive synthetic corpus for

pretraining was generated by prompting a Qwen3-32B LLM.

2. The model was fine-tuned on smaller high-quality supervised pair sets. This pair set is a curated mixture of labeled datasets (MS MARCO [11], Natural Questions [14], HotpotQA [43], among others) and filtered high-quality synthetic pairs from the initial corpus by normalization that incorporates a masking mechanism to filter false negatives by excluding pairs where similarity exceeds $s(q_i, d_i^+) + 0.1$ or exhibiting string equality with positive documents.

3. The final training stage applies a spherical linear interpolation (SLERP) model merging [44] across multiple checkpoints to enhance robustness and generalization across varying data distributions.

The training objective employs an improved InfoNCE-based contrastive loss function defined as

$$\mathcal{L}_{\text{embedding}} = -\frac{1}{N}\sum_{i}^{N} \log \frac{e^{(s(q_i, d_i^+)/\tau)}}{Z_i} \tag{3}$$

where $s(\cdot, \cdot)$ represents the cosine similarity, τ is a temperature parameter, and Z_i aggregates the similarities between positive pairs and various negative samples, including hard negatives, in-batch queries, and in-batch documents.

Table 2.2 summarizes the main characteristics and reported MTEB scores of the bi-encoder architectures discussed above.

TABLE 2.2
Comparison of Bi-Encoder Architectures: Main Characteristics and MTEB Performance

Model	Base Architecture	Ctx	MTEB Retrieval	MTEB Reranking	Key Innovation
SBERT	BERT-base	512	N/A	N/A	First efficient sentence embeddings via mean pooling
RoBERTa	BERT arch.	512	N/A	N/A	Dynamic masking, byte-level BPE
XLM-RoBERTa	RoBERTa	512	N/A	N/A	Cross-lingual transfer via multilingual pretraining
E5-base	BERT-base	512	37.69	41.49	Massive weak supervision with query/passage prefixes
mE5-large	XLM-RoBERTa-L	512	53.67	62.78	Multilingual capability
BGE-M3	XLM-RoBERTa-L + RetroMAE	8192	54.6	62.79	Unified dense/sparse/multi-vector; self-KD
Qwen3-0.6B	Qwen3-0.6B (causal LM)	8192	64.65	61.41	Causal attention with [EOS] pooling
Qwen3-8B	Qwen3-8B (causal LM)	8192	85.13	65.55	

2.2.3 Cross-Encoder Architectures

The Standard CE Reranking Paradigm The modern cross-encoder reranking paradigm was established by Nogueira and Cho [8], who demonstrated that BERT could be directly applied to passage reranking by concatenating the query and candidate passage as a single input sequence with a $[\mathrm{SEP}]$ separator. A linear classification head on the $[\mathrm{CLS}]$ token then produces a relevance score, and passages

are reranked by descending score. This simple formulation allowed all transformer layers to attend across both query and passage tokens simultaneously, capturing token-level interactions that bi-encoders cannot express.

Applied to the MS MARCO passage ranking benchmark, BERT-based reranking produced dramatic gains over BM25, establishing the cross-encoder as the go-to architecture for the reranking stage. The underlying principle—processing query-document pairs jointly rather than independently—remains the defining characteristic of every CE reranker discussed subsequently. The essential training recipe is also established here: the model is optimized with a pointwise binary cross-entropy loss on labeled relevant/non-relevant pairs, with hard negatives sourced from BM25 retrieval to make the discrimination task non-trivial.

Subsequent work extended this paradigm to sequence-to-sequence models. MonoT5 [45] reformulates relevance scoring as conditional text generation: the model receives a prompt containing the query and passage and is trained to produce the token "true" for relevant pairs and "false" for irrelevant ones. The log-probability of "true" serves as the relevance score. MonoT5 demonstrated that generative pretraining can effectively transfer to discriminative reranking, and its approach to treating reranking as a language modeling task influenced the design of subsequent LLM-based rerankers. However, because it requires autoregressive decoding for every candidate pair, inference latency is higher than for encoder-only CEs with the same parameter count.

GTE Multilingual Reranker is a pairwise CE within the mGTE [34] framework. Architecturally, it implements a 12-layer encoder-only Transformer with 768 hidden dimensions and 12 attention heads, optimized for efficiency through Rotary Positional Embeddings [46] that support 32k-token context lengths and de-padding

techniques to accelerate long-sequence processing. It processes query-document pairs individually in a pairwise fashion, concatenating inputs as [CLS] query [SEP] document [SEP], and deriving relevance scores from the final [CLS] token representation via a linear projection layer. The model was trained in two distinct stages.

The model is trained using binary cross-entropy loss ($\mathcal{L}_{\text{BCE}}$) on multilingual query-document pairs. Positive documents (d^+) are relevant to query q, whereas negatives (d^-) are hard negatives dynamically sampled from both BM25 and dense retrieval systems. The relevance score $s(q, d)$ is passed through a sigmoid function $\sigma(\cdot)$ with labels $y = 1$ for positives and $y = 0$ for negatives. This stage emphasizes robust negative discrimination across languages. Then, it undergoes fine-tuning on high-quality annotated datasets such as MS MARCO [11] and mMARCO [47] using a margin-based ranking loss ($\mathcal{L}_{\text{margin}}$).

$$\mathcal{L}_{\text{margin}} = \max\left(0, \Delta - s(q, d^+) + s(q, d^-)\right), \tag{4}$$

This loss enforces a margin $\Delta > 0$ between the scores of positive d^+ and negative d^- documents for the same query q, sharpening the model's ability to discriminate relevance at a finer granularity. The pairwise nature of both loss functions fits the CE design of the model, where each query-document pair is processed independently to compute precise interaction-based relevance scores.

Critical architectural choices include byte-level BPE tokenization supporting 100+ languages and asymmetric query/document processing, where document tokens are truncated to 512 tokens while queries retain their full context. This pairwise design enables precise token-level interaction between queries and documents, but limits the parallel processing of multiple candidates per inference pass, a deliberate

trade-off for higher reranking accuracy in multilingual settings. Evaluated on the MIRACL and Benchmarking Information Retrieval (BEIR) benchmarks, it achieves SOTA zero-shot reranking performance across 18 languages while maintaining an inference latency of under 200ms per pair on standard GPUs.

ListConRanker ListConRanker [48] is included here as a representative of the *listwise* training paradigm, which stands in contrast to the pointwise and contrastive objectives used in this thesis. Unlike the CE models described above, which process each query-passage pair independently, ListConRanker jointly encodes a query together with a full list of candidate passages via a specialized *ListTransformer* module. This architecture enables attention among all candidate passages simultaneously, allowing the model to learn relative similarity through direct comparison across the candidate set rather than through independent pairwise scoring.

Training uses Circle Loss [49], which simultaneously penalizes positive pairs scoring below an upper margin and negative pairs scoring above a lower margin, providing self-paced adaptive gradient weighting. The key practical benefit is that all candidates in the list contribute to a single forward pass, reducing inference calls compared to iterating over pairs. The tradeoff is that the candidate list must fit within the model's context window, constraining the number of passages that can be jointly reranked. Evaluated on Chinese retrieval benchmarks (C-MTEB [50], T2Reranking [51]), ListConRanker achieves strong listwise reranking performance. The contrast with this thesis is deliberate: the methodology here uses a Multiple Negatives Ranking Loss (an InfoNCE-style contrastive objective), which achieves list-level discrimination through in-batch and mined hard negatives rather than through joint encoding of the candidate list.

Qwen3 Reranker The Qwen3 Reranker [35] is the CE component within the Qwen3 Embedding series, built on pretrained Qwen3 transformer backbones [42]. In contrast to the bi-encoder architecture of the Qwen3 Embedding models, the reranker uses a CE paradigm: query-document pairs are jointly encoded through full cross-attention, allowing fine-grained token-level interaction. The architecture concatenates the query and candidate documents into a unified input sequence, processes them through Qwen3 transformer layers with causal self-attention, and derives the relevance score from the final representation layer. Unlike the Qwen3 bi-encoder model, the reranker omits a weakly supervised pretraining stage. The overall pipeline comprises two stages.

1. The model undergoes supervised fine-tuning on curated high-quality datasets.
2. The final training stage incorporates SLERP model merging [44].

The Qwen3 Reranker complements the bi-encoder models with full cross-attention between query and document, which yields higher relevance precision at the cost of per-pair inference.

ModernBERT and RuModernBERT ModernBERT [52] is a recently introduced family of encoder-only transformers that retains the BERT-style masked language modeling pretraining paradigm while incorporating a series of architectural improvements motivated by efficiency and context length. Key design changes include Rotary Positional Embeddings (RoPE) in place of learned absolute positions, Flash Attention for memory-efficient long-sequence processing, alternating local and global attention layers to reduce the quadratic cost of full self-attention, and an extended context window of up to 8,192 tokens. On long inputs it is significantly

faster than BERT-scale models, and it trains on modern hardware without specialized infrastructure.

RuModernBERT-base [53] is a Russian-language adaptation of ModernBERT, pretrained by DeepVK on a large Russian text corpus using the same architectural blueprint. Such language-adapted pretraining is motivated by the broader finding that continued domain- and task-adaptive pretraining yields measurable downstream gains over general-purpose multilingual baselines [54]. This makes it a well-suited backbone for the cross-encoder developed in this thesis: it provides a compact encoder (base-scale parameter count) with genuinely long-context capability and Russian-specific linguistic pretraining, while remaining efficient enough to serve as a practical CE in production-oriented scenarios. The model is used as the encoder backbone in this thesis, and its selection over larger multilingual alternatives reflects the pragmatic goal of developing a deployable Russian CE rather than simply maximizing benchmark scores at unbounded inference cost.

Together these architectures motivate cross-encoder reranking in practice: once a first-stage retriever has narrowed the candidate set, full query-document interaction can improve top-rank precision. RuModernBERT-base is chosen as the encoder backbone for a specific reason: it is a compact, Russian-pretrained, long-context encoder that can be deployed without the infrastructure overhead of billion-parameter models.

Architecture alone, however, is not sufficient. Nogueira and Cho's foundational CE work already showed that training data quality—in particular the difficulty and reliability of the negative examples—matters as much as the architecture. Modern rerankers trained on large English and multilingual corpora benefit from scale, domain diversity, and carefully constructed training pairs accumulated over years. For Russian, no comparable infrastructure exists, and what does exist is

fragmented across sources with different label semantics, quality levels, and domain coverage. The remaining sections therefore address the data side: what Russian resources are available, why their labels are noisy, and what techniques can mitigate that noise.

2.3 Datasets and the Russian Training-Data Gap

2.3.1 Methodological Role of Training Data

Cross-encoders are worth the inference cost when top-ranked precision matters more than raw throughput. A CE jointly encodes the query and candidate passage, enabling token-level interactions not available to first-stage retrievers. This capability matters only if training examples teach the model to distinguish between passages that are all superficially plausible. CE quality therefore depends not only on the backbone architecture but also on how positive and negative training signals are constructed.

Large-resource retrieval settings, including English, Chinese, and broad multilingual collections, benefit from large supervised or weakly supervised datasets, translated corpora, and mature benchmark ecosystems. The MS MARCO passage ranking dataset [11] remains a central source for reranker training, and mMARCO extends it to multiple languages through translation [47]. Chinese reranking resources and evaluations, such as T2Ranking and C-MTEB, similarly support dedicated reranking research [50], [51]. These resources show that strong reranking systems are often built not from a single modeling idea, but from a combination of scale, domain diversity, and carefully constructed training pairs. Representative examples are listed in Table 2.3.

TABLE 2.3
Examples of Reranking and Retrieval Resources Used in High-Resource Settings

Resource	Language Setting	Main Role	Relevance for CE Training
MS MARCO [11]	English	Passage ranking	Large weakly supervised source of query-passage pairs.
mMARCO [47]	Multilingual translation	Multilingual ranking data	Expands MS MARCO-style supervision, but introduces translation and adaptation noise.
BEIR [31]	Mostly English, multi-domain	Zero-shot evaluation	Tests domain generalization beyond web-search style data.
T2Ranking / C-MTEB [50], [51]	Chinese	Retrieval and reranking evaluation	Enables language-specific reranker development outside English.

2.3.2 Russian Evaluation Resources and Training-Data Fragmentation

Russian IR has recently gained stronger evaluation infrastructure. RusBEIR [10] provides a unified benchmark for zero-shot Russian retrieval across datasets adapted, translated, or created for different domains. ruMTEB [55] further extends the MTEB-style evaluation paradigm to Russian and includes retrieval and reranking tasks. These benchmarks make model comparison more systematic and reproducible.

The existence of evaluation benchmarks, however, does not by itself solve the training problem. Russian reranker training data remains heterogeneous: some sources are translated from other languages, some are adapted from question answering or reading comprehension, and some come from community-generated query-answer structures. This matters because CE training is sensitive to label

semantics. A QA-derived positive passage may contain the answer to a question, but it is not necessarily the best passage for ranking: it may contain only a fragment of the answer, include irrelevant context, or have many unlabeled alternatives that are also relevant. When such data is converted into retrieval format, unlabeled relevant passages may be treated as negatives during training.

RuBQ 2.0 [56] and SberQuAD [57] illustrate this trade-off. Both provide useful Russian question-context supervision, but their original objective is answer extraction or question answering rather than direct passage reranking. ru-MIRACL [58] provides higher-quality monolingual retrieval judgments for Russian, while community-sourced datasets offer volume but often contain weak, noisy, or domain-specific relevance signals. Russian CE development is mainly a data-quality and integration problem. Useful resources exist, but they need to be unified, filtered, and converted into training instances that actually reflect the reranking task. Table 2.4 summarizes the main Russian resources and their training challenges.

2.4 Weak Supervision, Noise, and Consistency Filtering

The three sections that follow address the data preparation pipeline underlying the methodology of this thesis. They form an ordered chain: the quality of noise filtering determines the reliability of hard negative mining, and the quality of hard negatives determines the informativeness of the contrastive training signal. For a low-resource language such as Russian, where training data is assembled from heterogeneous, partially noisy sources, each stage matters. A failure in the first stage (keeping noisy positives) propagates into the second (mining false negatives) and finally corrupts the third (contrastive learning from contradictory examples).

TABLE 2.4
Russian Retrieval Resources and Their Training Challenges

Resource	Original Task	Usefulness	Challenge for CE Training
RusBEIR [10]	Retrieval evaluation	Standardized Russian benchmark	Better suited for evaluation than unrestricted training because benchmark leakage must be controlled.
ruMTEB [55]	Multi-task evaluation	Retrieval and reranking comparison	Requires careful separation from training data when used for final comparison.
RuBQ 2.0 [56]	KBQA / MRC	Native Russian question-context pairs	Answer-bearing context is not always equivalent to optimal reranking relevance.
SberQuAD [57]	Reading comprehension	Larger Russian QA supervision	QA labels can produce weak positives and unlabeled relevant passages.
Community data	Forums / Q&A / web sources	Scale and domain diversity	Noisy positives, heterogeneous domains, and incomplete relevance labels.

Weak supervision is common in retrieval training because exhaustive human relevance annotation is expensive. In reranker training, however, weak positives are especially risky. If a labeled positive is not genuinely aligned with the query, then the training objective pushes the CE toward an incorrect relevance decision. The problem becomes more severe before hard negative mining: a noisy positive can distort the neighborhood from which difficult negatives are selected, producing contradictory training examples.

Prior work on dense retrieval emphasizes the importance of denoising and training-data quality. RocketQA, for example, combines hard negatives and

denoising strategies to improve retrieval training under imperfect supervision [59]. Multilingual and translated retrieval data introduce additional sources of noise: mMARCO makes multilingual MS MARCO-style training possible, but translation can change wording, specificity, and query-passage alignment [47]. Similar concerns are reported in multilingual BEIR-style settings, where translated corpora may differ from native retrieval distributions [60].

These findings motivate consistency filtering as a preprocessing step. A training pair should not be accepted solely because it appears in a source dataset; it should also be consistent with an independent retrieval signal. If a frozen retriever cannot place the labeled positive near the query within the relevant passage pool, the pair is likely noisy or poorly aligned for reranker training. Filtering such pairs before negative mining reduces the probability that the CE learns from corrupted positives and that subsequent hard negatives are selected around unreliable anchors.

2.5 Hard Negative Mining and False-Negative Risk

Hard negative mining is central to reranker training because CEs are normally applied after first-stage retrieval, where candidates are already plausible. Random negatives may be too easy: they often differ strongly from the query and therefore do not teach the model the fine-grained distinctions that justify using a CE. Hard negatives, by contrast, are lexically or semantically close to the query but labeled as irrelevant, forcing the model to learn a sharper decision boundary [61].

This requirement is stronger for CEs than for first-stage retrieval models. A bi-encoder must learn a broad embedding geometry suitable for approximate search over large collections. A CE is usually responsible for the final ordering of tens or hundreds of candidates returned by that first stage. Its training set

should therefore include cases that resemble the reranking situation: passages that first-stage retrieval would plausibly surface, but that differ in true relevance. This is why many reranking pipelines use BM25 or dense retrievers to mine difficult negatives [8], [59].

The benefit of hard negatives comes with a known risk: some mined negatives may actually be relevant but unlabeled. This false-negative problem is common when relevance labels are incomplete, and it is amplified in QA-derived or multilingual datasets where several passages can answer the same query. In cross-lingual settings, translated variants of the same conceptual positive can also be incorrectly sampled as negatives unless all known positive passages are excluded. A mixed composition of mined hard negatives and random or in-batch negatives can reduce this risk while retaining difficult contrastive examples [61]. The false-negative problem has also motivated debiased contrastive objectives that explicitly correct for the probability of sampling a true positive as a negative [62].

2.6 Contrastive Training and Effective Batch Size

Rerankers can be trained with pointwise classification losses, pairwise margin losses, listwise objectives, or contrastive objectives. The methodology of this thesis uses a Multiple Negatives Ranking Loss (MNRL), which is an InfoNCE-style objective: a query-positive pair is contrasted against multiple negatives, and the model is optimized to assign the positive the highest score [63], [64]. This objective fits reranking well because it directly trains the model to prefer the relevant passage among competing candidates.

The quality of the negative set matters a lot for contrastive training. In-batch negatives increase the number of comparisons without requiring explicit annotation

for every pair, while mined hard negatives increase the difficulty of the comparison. For CEs, memory is a practical limitation because each query-passage pair requires a transformer forward pass. Gradient caching addresses this limitation by decoupling representation computation from loss computation, which allows larger effective contrastive batches under fixed GPU memory [65]. In this thesis, this idea supports training with a larger set of negative comparisons than would otherwise fit into memory.

A complementary post-training technique is Spherical Linear Interpolation (SLERP) of model checkpoints [44]. Rather than averaging parameters directly, SLERP interpolates along the geodesic on the unit hypersphere defined by two checkpoint weight vectors, preserving parameter norms in a way that standard weighted averaging does not. This technique has been applied to merge checkpoints trained under different data conditions or at different training stages—as seen in both the Qwen3 Embedding and Qwen3 Reranker pipelines described above—to obtain a merged model with better generalization across data distributions. In this thesis, SLERP is used as a stabilization mechanism for the final checkpoint, not as a remedy for Russian data scarcity.

2.7 Cross-Lingual Augmentation and Distillation Alternatives

Multilingual encoders such as mBERT and XLM-RoBERTa demonstrate that shared representations can transfer knowledge across languages [3], [33]. In retrieval, this property enables models trained on high-resource data to generalize to lower-resource languages, especially when parallel or comparable corpora align semantically equivalent texts. LaBSE is a representative example of using

multilingual contrastive training to align translated sentences in a shared embedding space [66].

For the present thesis, cross-lingual information is used more narrowly. The goal is not to train a broad multilingual reranker that treats Russian as one language among many. Instead, the goal is to train a Russian-focused CE that remains robust when training data contains translated pairs, mixed-language query-passage combinations, or English domain-specific examples. Cross-lingual augmentation is therefore treated as a controlled data-construction technique rather than as the central source of model capability.

Knowledge distillation is worth noting as an alternative for compact reranker training. A strong teacher CE or LLM can provide soft relevance scores to train a smaller model, and cross-lingual distillation can transfer ranking behavior across languages [67]. However, distillation also transfers teacher biases and can obscure whether performance gains come from the student's own data pipeline or from the teacher. In benchmark-driven settings, teacher models may also have unknown exposure to public test data. For these reasons, this thesis focuses on data preparation, contrastive training, and controlled negative construction rather than making teacher distillation the main mechanism.

2.8 Evaluation Integrity and Benchmark Contamination

The use of public benchmark data for training, while common in model development, must be controlled when making comparative claims. Benchmark Data Contamination occurs when a model is exposed to evaluation examples during pretraining, fine-tuning, or data selection, producing scores that overestimate true

generalization [68], [69]. This risk is particularly difficult to exclude for large multilingual foundation models trained on web-scale corpora.

For a Russian CE, the practical evaluation problem is narrower but still real: training sources must be checked against the sources used to construct held-out evaluation datasets, and final test sets should not be used during filtering, mining, or model selection. Documentation of training data provenance is therefore part of the methodology, not merely an implementation detail [70]. This motivates the thesis protocol of separating train/validation data from out-of-distribution test sets and tracking dataset lineage when comparing against external rerankers.

2.9 Open Challenges and Positioning

2.9.1 Data Quality and Russian-Specific Supervision

The central bottleneck for Russian CE training is not the complete absence of data, but the mismatch between available supervision and the reranking objective. Translated resources, QA-derived corpora, community data, and retrieval benchmarks all contribute useful signals, but each source has different label semantics and noise patterns. The pipeline therefore first unifies data, then filters for consistency, and only then mines hard negatives for contrastive CE training.

2.9.2 Compact Cross-Encoders and Inference Cost

The computational cost of CEs remains a major practical limitation. Current strong rerankers often operate at several hundred million parameters, such as Qwen3-0.6B and BGE-M3-scale models [35], [40], while LLM-based rerankers can reach billions of parameters. Larger models may improve ranking quality, but the inference

cost grows substantially because every candidate pair requires a forward pass. For deployment-oriented reranking, several-hundred-million-parameter models form a practical competitive frontier, and even smaller models are valuable if careful specialization can narrow the performance gap.

The model choice in this thesis should therefore be interpreted pragmatically. ModernBERT-family encoders provide efficient long-context transformer backbones [52], and RuModernBERT-base offers a compact Russian-focused instance [53]. The thesis does not claim that this backbone is globally optimal for Russian reranking. Rather, it tests whether a compact Russian-focused CE, trained with a data-centric pipeline, can approach much larger multilingual rerankers while remaining easier to train and deploy.

2.9.3 Synthesis: Positioning of This Thesis

The reviewed literature establishes CEs as a strong architecture for high-precision reranking, but it also shows that model performance depends heavily on training data quality and negative construction. Existing multilingual rerankers benefit from broad high-resource mixtures and large model capacity, whereas Russian reranker training must integrate fragmented, translated, QA-derived, and weakly supervised data. This thesis addresses that gap by studying a compact Russian-focused CE trained through data unification, consistency filtering, hard-negative mining, cross-lingual augmentation, MNRL-style contrastive training, and controlled evaluation against strong external rerankers.

Chapter 3

Methodology

3.1 General Architecture of the Proposed Pipeline

The methodology describes an end-to-end pipeline that builds a training corpus and trains a Russian cross-encoder reranker on top of it. The design follows prior work on neural passage reranking [8] and data-centric optimization for dense retrieval [59].

The pipeline comprises seven sequential stages:

1. **Unification** (§3.2.1). Source corpora from heterogeneous origins are mapped to a common schema and cleaned to produce a single collection of query–passage pairs.

2. **Splitting** (§3.2.2). The unified collection is partitioned into training and validation subsets. Splitting is performed at the level of connected components in the query–passage graph, so no query or passage text can appear in more than one partition.

3. **Embedding**. A frozen dense retrieval model $\mathcal{E}$ encodes all queries and

passages in the training and validation partitions. The resulting per-dataset embedding indices are used by both the filtering and mining stages that follow.

4. **Consistency Filtering** (§3.2.3). Each training pair is evaluated against $\mathcal{E}$: a pair is kept only if the labeled positive passage can be retrieved by $\mathcal{E}$ near the top of the dataset-local passage pool. This step removes annotation noise before it can corrupt negative mining.

5. **Hard Negative Mining** (§3.2.4). For each retained query, passages that lie close to the query in $\mathcal{E}$'s embedding space but are not labeled as positives are selected as hard negatives. These are the examples that will most challenge the cross-encoder during training.

6. **Cross-Lingual Augmentation and Compilation** (§3.2.5). For multilingual datasets, cross-lingual query–passage combinations are generated and merged with the mined negatives to form the final training corpus.

7. **Training and Validation** (§3.3). The cross-encoder is fine-tuned on the compiled corpus using a contrastive ranking loss. Model quality is tracked on the validation split throughout training, and the checkpoint with the highest validation MAP is retained.

The retrieval model $\mathcal{E}$ is central to Stages 3–5: it governs which pairs survive filtering and which passages qualify as hard negatives. Its choice therefore shapes the entire downstream training signal; the specific model used is reported in §4.2.3. One ordering constraint is critical: filtering must be applied *after* the data split (Stage 4 after Stage 2). Filtering before splitting would change the query distribution from which the validation set is drawn, biasing that partition toward queries that $\mathcal{E}$ retrieves more easily.

3.2 Data Processing

3.2.1 Unification and Preparation of the Source Data

The source corpora were originally distributed in different formats with different field conventions. For uniform processing, all datasets were mapped to a common schema with the fields query, passage, dataset, lang, query_id, and passage_id. The dataset and lang fields preserve dataset provenance and linguistic metadata, enabling dataset-aware batching (§3.3.2) and language-consistent negative selection (§3.2.4).

For sources lacking native identifiers, stable BLAKE2b hashes of the concatenated textual content were generated as surrogate IDs. BLAKE2b was selected for its collision resistance and determinism: the same textual instance always produces the same 64-bit digest across independent pipeline runs, maintaining an unambiguous mapping between textual instances and their query–passage relations.

The preparation stage further encompassed the following cleaning operations, applied in the stated order:

1. **Type enforcement**: all schema fields are cast to their declared types (str for text and identifier fields); rows failing this cast are discarded.

2. **Null and empty-string removal**: rows in which any of query, passage, query_id, or passage_id are null, empty, or contain only whitespace after stripping are dropped.

3. **Unicode normalization**: all text fields are normalized to NFC (Normalization Form C), the Unicode standard form that canonically decomposes and then recomposes characters, eliminating encoding differences between otherwise identical strings.

4. **Minimum-length filtering**: pairs in which the query or passage contains fewer than three Unicode code points after stripping are discarded as degenerate.

Serialization uses the Apache Parquet columnar format with Snappy compression. This enables out-of-core processing at scale through column-selective reads and memory-mapped access, without loading entire files into RAM.

3.2.2 Formation of Training and Validation Splits

Splitting is defined at the level of query–passage pairs identified by their stable ID columns, not at the level of isolated rows. For parallel corpora, where the same conceptual pair is replicated across multiple languages, a completeness constraint is applied so that all language variants of a pair are assigned to the same partition.

To prevent information leakage, the dataset is modeled as a bipartite graph $G = (Q \cup P,\ E)$ in which query identifiers Q and passage identifiers P form two disjoint node sets, and each observed pair constitutes an edge $e \in E$. Connected components of G are treated as atomic units during splitting: any pair sharing a query or passage identifier with another pair—either directly or transitively—is assigned to the same partition. This bipartite-component criterion guarantees that no query and no passage text appears in more than one split, preventing both identifier and content leakage.

The components are shuffled using a fixed random seed and assigned to training and validation partitions in approximate proportions of $95:5$. Exact proportions are not guaranteed owing to variability in component sizes; priority is given to preventing leakage over achieving precise ratios. Per-dataset split statistics are reported in Table 4.5.

3.2.3 Filtering Training Pairs by the Consistency Criterion

To mitigate label noise inherent in weakly supervised and machine-translated corpora, training pairs undergo a consistency-filtering procedure prior to hard negative mining. The rationale for this ordering is that noisy positive labels corrupt the hard-negative mining step: if a labeled positive is not genuinely relevant, passages retrieved nearby it in embedding space may be mined as false negatives, introducing contradictory gradient signal into training [59]. Moreover, in multilingual and translated corpora, additional noise arises from imperfect translation, which has been shown to degrade retrieval quality [47], [60].

Notation and rank statistics. Let $\mathcal{E}$ denote the dense retrieval model and $\mathcal{P}_d$ the set of passage identifiers belonging to dataset d. For a query q and its labeled positive passage $p^+ \in \mathcal{P}_d$, two rank statistics are computed and stored as output columns:

- $\mathrm{retrieval_rank}(q,\, p^+)$: the 0-indexed rank of p^+ in the list returned by a nearest-neighbor search of $\mathcal{E}$'s embedding index over $\mathcal{P}_d$. If p^+ is not found within the top-$k_{\max}$ results, the value is recorded as -1.

- $\mathrm{virtual_rank}(q,\, p^+,\, D^+)$: the same rank recomputed after temporarily removing all other labeled positives $D^+ \setminus \{p^+\}$ from the candidate list ("hole-fitting"). This corrects for multi-positive queries where multiple correct answers cluster in embedding space and suppress each other's retrieval rank.

Filtering rule. A training pair $(q,\ p^+)$ is retained if and only if:

$$\begin{aligned}&\text{retrieval_rank}(q, p^+) < k \\ &\qquad \vee\ \big(\text{retrieval_rank}(q, p^+) = -1\ \wedge\ \text{virtual_rank}(q, p^+, D^+) < k\big)\end{aligned} \quad (5)$$

where k is a dataset-relative threshold whose specific values are reported in §4.2.2. The virtual rank serves as a fallback: if the primary rank fails (value $= -1$), the pair is still retained if p^+ is retrievable once competing positives are excluded. Both statistics are preserved as output columns to support post-hoc analysis of filtering attrition.

Passage pool. The FAISS index [71] used for filtering is built per-dataset from $\mathcal{P}_d$ rather than from a global corpus index. This choice is motivated by computational tractability at scale: maintaining and querying a single global index over the full multi-source passage corpus is infeasible within the pipeline's memory budget. A consequence is that the filtering criterion is *dataset-relative*: retrieval_rank depends on the density and homogeneity of the local passage pool. High-attrition datasets (see Table 4.2) indicate that a large fraction of their positive pairs are not retrievable by $\mathcal{E}$ within the dataset-local pool—consistent with the presence of indirect or paraphrastic relevance judgements that do not manifest as high-similarity pairs under $\mathcal{E}$'s representation function. Attrition rates are therefore not comparable across datasets.

Limitation of the hole-fitting strategy. Because removing competing positives from the index can only reduce the target passage's rank (move it higher in the list), the virtual rank is always $\leq$ the retrieval rank. The fallback criterion is therefore strictly more permissive: some pairs that $\mathcal{E}$ cannot retrieve even without competition

pass the virtual-rank gate and remain in training. These pairs represent the noisiest subset of the retained data and their contribution should be accounted for during result analysis.

3.2.4 Hard Negative Mining

Hard negative mining introduces semantically challenging negative examples that are difficult to discriminate from the query's genuine positives. Without such examples, the contrastive objective primarily learns to separate queries from clearly irrelevant passages, providing an insufficient training signal for high-precision reranking.

Candidate retrieval and adaptive depth. For each query q, a pool of candidate negative passages is retrieved using $\mathcal{E}$'s dataset-local FAISS index (the same index as used in filtering). An initial search depth of k_0 is used; if fewer than the required minimum $N_{\min}$ valid negatives are found in this pool, the search depth is doubled iteratively up to a maximum of $k_{\max}$. Queries for which fewer than $N_{\min}$ valid negatives can be identified even at $k_{\max}$ are discarded from training. Specific values for k_0, $k_{\max}$, and $N_{\min}$ are reported in §4.2.3.

Margin-based selection criterion. Given the retrieved pool, a per-query score threshold $\tau(q)$ is defined relative to the query's same-language positive passages:

$$\tau(q) \;=\; \bar{s}(q,\, D^{+}_{\text{same}}) \;-\; \left|\bar{s}(q,\, D^{+}_{\text{same}})\right| \cdot \delta \tag{6}$$

where $\bar{s}(q,\, D^{+}_{\text{same}})$ is the mean inner-product similarity between q and its same-language positive passages, and δ is the relative margin parameter. A candidate

passage d is selected as a hard negative if $s(q, d) \leq \tau(q)$: the passage is similar to the query, but falls below the mean positive similarity by at least a fraction δ. Using same-language positives as the reference prevents the margin from being distorted by cross-lingual pairs, which may occupy a different absolute similarity scale. The specific value of δ is reported in §4.2.3.

This relative formulation is preferred over an absolute threshold because the absolute scale of inner-product similarities varies across queries, datasets, and embedding models. A relative margin adapts to each query's own similarity distribution, providing consistent hardness semantics regardless of the overall score range.

Cross-lingual positive exclusion. All known positive passages for a given base query—including cross-lingual variants sharing the same base query identifier—are excluded from the negative candidate pool before selection. This cross-lingual exclusion prevents false negatives: a Russian passage that correctly answers the same conceptual question as an English positive would, without this rule, be mined as a hard negative for the English query variant, producing a contradictory training signal.

Retrieval model bias. The notion of hardness is defined relative to $\mathcal{E}$: passages that are confusable under $\mathcal{E}$'s embedding function are designated as hard negatives. Domain skews or language-pair asymmetries in $\mathcal{E}$'s representation space therefore propagate into the training signal of the cross-encoder [59], [61]. This is a known limitation of all retrieval-based hard negative mining pipelines.

3.2.5 Cross-Lingual Data Augmentation and Final Dataset Compilation

For multilingual source datasets that provide paired translations (i.e., the same conceptual query–passage pair in both Russian and English), additional training instances are constructed by forming cross-lingual combinations. Given a Russian query q_{ru}, English query q_{en}, Russian passage p_{ru}, and English passage p_{en}, four instances are generated: the two monolingual pairs $(q_{\text{ru}}, p_{\text{ru}})$ and $(q_{\text{en}}, p_{\text{en}})$, and the two cross-lingual pairs $(q_{\text{ru}}, p_{\text{en}})$ and $(q_{\text{en}}, p_{\text{ru}})$.

This augmentation, applied to datasets labeled en+ru in Table 4.5, is responsible for the approximately twofold expansion of pair counts observed for those datasets between the filtered and final columns. Cross-lingual instances expose the cross-encoder to mixed-language inputs, encouraging language-agnostic relevance representations rather than surface-level lexical matching within a single language.

Hard negatives are selected in a language-consistent manner: a query in language ℓ is paired with negatives mined within the same language's passage pool. This preserves the notion of semantic hardness as defined by $\mathcal{E}$ within a single language and prevents trivial lexical discrimination (e.g., rejecting a passage solely because it is in a different language from the query).

The final training instances are materialized as columnar Parquet files with the schema $\{\text{query}, \text{positive}, \text{negative_1}, \ldots, \text{negative_N}\}$, where each row contains one positive passage and up to N hard negative passages per query. Positives are grouped by base query identifier so that all language variants of the same conceptual query share a common positive set, which is used for false-negative exclusion during both training (§3.3.3) and evaluation (§3.4).

3.3 Training Procedure

3.3.1 Architecture Selection Criteria

The base model for all experiments is RuModernBERT-base, a 150M-parameter encoder pre-trained specifically on Russian text using the ModernBERT architecture [52]. It was selected on the basis of the following criteria:

- **Russian linguistic competence**: RuModernBERT-base is pre-trained and tokenized specifically for Russian, providing dense vocabulary coverage of inflected word forms and reducing subword fragmentation relative to multilingual models [12].
- **Extended context support**: the ModernBERT architecture natively supports sequences up to 8,192 tokens, substantially exceeding the 512-token limit of early BERT-family encoders [3] and accommodating passages of typical encyclopedia-article or legal-document length.
- **Parameter efficiency**: at 150M parameters, the model fits within the VRAM of a single H200 device, enabling large effective batch sizes through gradient caching and systematic ablation studies on a fixed hardware budget.
- **Established reranking capability**: the ModernBERT architecture has demonstrated competitive performance on cross-encoder reranking benchmarks, providing a solid foundation without the need for architectural modifications.

The exact training configuration for RuModernBERT-base is described in §4.3.

3.3.2 Batching and Sampling Strategy

The training procedure employs a dataset-aware batching strategy to improve computational efficiency in multi-GPU settings with heterogeneous data sources. At each optimization step, all devices process mini-batches drawn exclusively from the same source dataset.

This design is motivated by sequence-length heterogeneity across datasets: passage lengths vary substantially by source corpus, with some concentrated almost entirely in the shortest bucket and others containing a much larger share of long passages (empirically verified in Figure 4.1, presented alongside the dataset descriptions in §4.2). Query lengths are more similar across datasets, so passage length is the main driver of padding variability. Since transformer batches are padded to the longest sequence in the mini-batch, mixing datasets with different long-passage rates increases the probability that a small number of very long passages force excessive padding for many short passages. Dataset-aware batching reduces this padding overhead by forming batches from samples with more similar length distributions.

The experiments in this thesis were run on homogeneous GPUs, so hardware-level straggler effects from faster and slower devices are not the main concern. Nevertheless, keeping all ranks on the same dataset at each step also prevents different ranks from processing batches with systematically different sequence-length profiles, which helps keep padding behavior more predictable during synchronous distributed training.

Cross-rank schedule synchronization. To guarantee that all ranks process the same dataset at every gradient step, the following synchronization protocol is applied at the start of each epoch:

1. Each rank independently computes its local number of available batches per source dataset.

2. All ranks perform an $\mathrm{all_reduce(MIN)}$ over these per-dataset counts to identify the globally minimum batch count for each dataset—i.e., the number of batches that every rank can provide.

3. A global dataset schedule is built from these minimum counts and identically shuffled on all ranks using a shared epoch-level random seed.

4. Each rank yields batches by following this shared schedule, drawing from its own rank-local batch lists.

This protocol requires disabling Accelerate's built-in batch-sampler sharding ($\mathrm{even_batches} = \mathrm{False}$, $\mathrm{dispatch_batches} = \mathrm{False}$), which would otherwise re-shard the custom sampler and break the cross-rank schedule synchronization. A post-construction integrity check verifies that the DataLoader's reported length matches the synchronized schedule length; a mismatch raises a runtime error before the first training step.

3.3.3 Loss Function and Negative Composition

Cross-encoder training employs the Multiple Negatives Ranking Loss (MNRL) with a fixed temperature scale factor, defined as:

$$\mathcal{L}_{\mathrm{MNRL}} = -\log \frac{\exp\big(\lambda \cdot s(q, d^+)\big)}{\exp\big(\lambda \cdot s(q, d^+)\big) + \sum_{k=1}^{K} \exp\big(\lambda \cdot s(q, d_k^-)\big)} \tag{7}$$

where $s(q, d)$ denotes the cross-encoder relevance score for query–passage pair (q, d), λ is a fixed scale factor, and $d_1^-, \ldots, d_K^-$ are the K negative passages

associated with the training instance. This objective is equivalent to a $(K+1)$-class softmax and directly optimizes ranking quality [63], [64]. The specific value of λ is reported in §4.3.1.

Negative composition. The K negatives per training instance comprise two complementary sources:

- K_H **hard negatives**: passages retrieved in the hard negative mining step (§3.2.4), stored as fixed columns $\text{negative_}1, \ldots, \text{negative_}K_H$ in the training dataset.
- K_R **random negatives**: passages sampled dynamically during training from the positive passages of other instances in the current contrastive mini-batch. Because the dataset-aware batch sampler (§3.3.2) guarantees that no query or passage appears more than once per batch, including cross-lingual variants of the same conceptual pair, the probability of a randomly sampled in-batch positive being a true positive for another query is negligible. This structural guarantee reduces the risk of false-negative contamination from random negatives compared to sampling from an unconstrained corpus pool.

In the ablation study, K_R is held fixed while K_H is varied, so that differences in loss behavior are attributable to the number and quality of hard negatives rather than to the random negative count. Training exclusively on hard negatives amplifies the risk of false hard-negative contamination: passages that are semantically relevant to the query but absent from the positive labels due to annotation incompleteness. Such passages receive contradictory gradient signal if treated as negatives. The mixed strategy mitigates this bias while retaining the discriminative benefit of semantically challenging examples [61].

Gradient caching. To decouple the effective contrastive batch size from GPU memory constraints, gradient caching [65] is employed. This technique partitions the contrastive batch into sub-batches for the forward and backward passes, accumulating exact gradients before the optimizer step. Gradient caching enables a large number of in-batch negatives visible to the loss function without requiring that all of them fit into device memory simultaneously.

3.3.4 Training Validation Metrics

During training, the model is evaluated on the internal validation split at regular intervals, producing approximately 20 evaluation points per epoch. The high evaluation frequency is motivated by the known sensitivity of cross-encoder fine-tuning: transformer-based rerankers can reach their best validation performance within the first epoch and subsequently plateau or regress, as reported by Nogueira and Cho [8], who observed no improvement in development set performance beyond a fraction of the training data. This behaviour is further consistent with the documented instability of BERT fine-tuning under gradient-based optimization [72]. Frequent checkpointing therefore reduces the risk of missing the performance peak.

The primary metric used for checkpoint selection is Mean Average Precision (MAP), defined as:

$$\mathrm{MAP} = \frac{1}{|Q|} \sum_{q \in Q} \mathrm{AP}(q), \qquad \mathrm{AP}(q) = \frac{1}{R_q} \sum_{r=1}^{N} P(r) \cdot \mathrm{rel}(r) \tag{8}$$

where R_q is the number of relevant passages for query q, $P(r)$ is the precision at rank r, and $\mathrm{rel}(r) \in \{0, 1\}$ indicates whether the r-th ranked passage is relevant. MAP is computed over the full ranked list and is used as the checkpoint selection criterion because it rewards placing all relevant passages high in the ranking, making

it sensitive to errors at any rank position. Normalized Discounted Cumulative Gain (NDCG) is not used for checkpoint selection because the internal validation set has binary relevance labels with one positive per query in most source datasets, under which NDCG provides no additional discriminative power over MAP. NDCG is instead used as the primary metric during final benchmark evaluation (§3.4).

Complementary metrics reported at each validation step include:

- $\mathrm{MRR@}k$: Mean Reciprocal Rank at cutoff k, defined as:

$$\mathrm{MRR@}k = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\mathrm{rank}_q} \tag{9}$$

  where rank_q is the position of the first relevant passage for query q within the top-k results, or ∞ if no relevant passage appears. MRR captures early precision by rewarding models that place the first correct answer as high as possible, and is particularly informative for single-positive-per-query datasets. Reported at k=10.

- $\mathrm{Accuracy@}k$: the fraction of queries for which at least one relevant passage appears within the top-k ranked results. Reported at $k \in \{1, 3, 5\}$, it provides a coarse but intuitive measure of how often the model places a correct answer near the top.

These metrics are computed on the internal validation split (§3.2.2), not on external benchmarks, and are used solely for monitoring training dynamics and checkpoint selection.

3.4 Evaluation Protocol

3.4.1 Data Isolation and Anti-Contamination Measures

A critical requirement for trustworthy evaluation of reranking models is strict separation between training and evaluation data, both at the identifier level and at the semantic level. Benchmark Data Contamination—the inadvertent exposure of a model to test data during training or fine-tuning—can produce artificially inflated performance scores that do not reflect the model's true generalization capacity [68], [69].

The evaluation protocol applies the following isolation measures in order of strictness:

Each candidate evaluation dataset is analyzed with respect to its construction methodology and known overlap with training corpora — both this work's and those of competitive baselines. Datasets whose passage corpus directly overlaps with a training source are excluded from evaluation entirely. Datasets exhibiting only corpus-level overlap (shared underlying text collection but distinct queries and relevance annotations) are retained with explicit documentation of the contamination risk. The training partition is defined before any interaction with evaluation data, ensuring that evaluation distributions are not shaped by filtering or mining decisions.

3.4.2 Two-Stage Evaluation Pipeline

Evaluation follows the standard two-stage retrieval–reranking pipeline used in IR benchmarking [31]. In the first stage, a fixed BM25 retriever selects the top-100 candidate passages per query from the full benchmark corpus. In the second stage, the reranker under evaluation scores each query–passage pair among those 100

candidates and produces a final ranked list. Evaluation metrics are computed over this reranked output.

This setup isolates the reranker's contribution from first-stage retrieval quality: because the BM25 candidate pool is fixed and identical for all evaluated models, any difference in NDCG@10 or other ranking metrics reflects exclusively the reranker's ability to order relevant passages near the top of the list. Fixing the first stage also mirrors real deployment conditions, where a lightweight retriever feeds a more expensive reranker.

3.4.3 Benchmark Coverage and Aggregation

Final evaluation is conducted on the RusBEIR benchmark suite [10], a collection of Russian-language IR and question-answering datasets spanning multiple domains and task types. Evaluating on a diverse multi-domain suite rather than a single benchmark is motivated by the generalization claim of the proposed approach: a reranker trained on a heterogeneous corpus should perform well across domains, not merely the ones best represented in training.

The aggregate score is computed as the unweighted average of per-dataset NDCG@10 across all active evaluation datasets. Equal weighting assigns each domain the same contribution to the final score regardless of the corpus size or number of queries in that dataset, preventing large benchmarks from dominating the aggregate and ensuring that gains on specialized domains (e.g., biomedical or financial) are equally reflected in the headline number.

3.4.4 Benchmark Evaluation Metrics

Final evaluation on held-out benchmarks uses a different metric set from the training-time validation. The primary headline metric is NDCG@10, following the conventions of TREC Deep Learning [73], BEIR [31], RusBEIR [10], and ruMTEB [55]. NDCG@10 measures ranking quality by summing relevance scores discounted logarithmically by rank position, normalized against the ideal ranking:

$$\mathrm{NDCG@}k = \frac{\mathrm{DCG@}k}{\mathrm{IDCG@}k}, \qquad \mathrm{DCG@}k = \sum_{r=1}^{k} \frac{\mathrm{rel}(r)}{\log_2(r+1)} \tag{10}$$

where $\mathrm{rel}(r)$ is the relevance grade at rank r and $\mathrm{IDCG@}k$ is the DCG of the ideal ranking. NDCG is preferred over MAP as the primary benchmark metric for two reasons: it is the standard in BEIR and ruMTEB, enabling direct comparison with published baselines; and it is defined at a fixed cutoff, making scores comparable across benchmarks with varying numbers of relevant passages.

Secondary benchmark metrics reported alongside NDCG@10 are MAP@10, MRR@10, and Precision at cutoff ($\mathrm{P@}k$, $k \in \{1, 3, 5\}$), where $\mathrm{P@}k$ is the fraction of the top-k retrieved passages that are relevant, averaged over queries.

3.4.5 Ablation Study Design

Four ablation studies are conducted to isolate the contributions of the primary methodological choices. In each case, a single independent variable is varied while all other components are held fixed.

Ablation A — Consistency filtering (§4.5): The full unfiltered training corpus (V1) is compared against the filtered corpus (V2), with the model architecture, negative sampling strategy, and all hyperparameters held fixed. The hypothesis

is that filtering improves reranking quality by reducing mislabeled positives and their downstream effect on hard negative quality.

Ablation B — Hard and random negative sampling (§4.6): K_R is fixed at 2 and K_H is varied across three conditions: the random-only baseline (K_H=0, K_R=2), and two mixed regimes (K_H=8, K_R=2) and (K_H=15, K_R=2), with all other components held fixed. The hypotheses are twofold: (i) in-batch random negatives alone are insufficient for high-precision reranking but serve as a useful regulariser when combined with hard negatives; and (ii) hard negatives provide a stronger discriminative training signal, but raising K_H too far increases false-negative contamination risk and yields diminishing returns.

Ablation C — SLERP checkpoint merging (§4.7): Two merging regimes are evaluated using the $\mathrm{multi_slerp}$ generalization of Spherical Linear Interpolation, which interpolates jointly across more than two weight vectors on the unit hypersphere in a single step. The first regime merges the best checkpoints of the three consistency-filtering runs (V1, V2, V3), testing whether SLERP can combine models trained on different data into a single reranker. The second regime merges several consecutive checkpoints from a single training run, testing whether SLERP over nearby optima smooths out stochastic variation in the optimization trajectory. In both regimes the merged model is compared against the single best constituent checkpoint.

3.5 Methodology Summary

The pipeline has seven stages: data unification, bipartite-component splitting, embedding-based consistency filtering, hard negative mining, cross-lingual augmentation, dataset compilation, and model training. Each addresses a specific failure mode of weakly supervised retrieval corpora.

A single dense retrieval model $\mathcal{E}$ is used as the backbone of both Stage 4 (consistency filtering) and Stage 5 (hard negative mining). Pairs that survive filtering are by construction ones whose positive passages $\mathcal{E}$ would retrieve, so the negatives mined around them are genuinely confusable rather than random. The cost is that biases in $\mathcal{E}$'s representation space propagate through both stages.

The system-level choices target scalability: columnar Parquet storage with per-dataset sharding for out-of-core processing, GPU-accelerated nearest-neighbor search with adaptive depth widening, and a dataset-aware batch sampler with cross-rank schedule synchronization for distributed training. The mixed hard–random negative composition trades discriminative power against false-negative contamination risk.

Experimental validation of the individual design choices is presented in Chapter 4.

Chapter 4

Experiments

4.1 Software and Computational Environment

All experiments were conducted within a Kubernetes-managed pod to ensure strict environment reproducibility and resource isolation across runs. The computational infrastructure comprised eight NVIDIA H200 GPUs, each equipped with 143,771 MiB of VRAM, yielding an aggregate GPU memory capacity of approximately 1.1 TiB. The host system provided 256 GiB of system RAM and 16 CPU cores. The operating system was Ubuntu 22.04 LTS with CUDA 13.1 serving as the GPU compute runtime. Two isolated Python virtual environments were maintained to address an irreconcilable dependency conflict between the training and indexing stages of the pipeline.

The *training environment* used Python 3.11.15 and was built around PyTorch 2.8.0, sentence-transformers 5.2.2, transformers 4.57.5, and flash-attn 2.8.3. The sentence-transformers framework was selected for its native support for cross-encoder fine-tuning, the Multiple Negatives Ranking Loss, and gradient caching — all three of which are central to the procedure described in Chapter 3.

A separate *indexing environment* based on Python 3.13.5 was dedicated to GPU-accelerated approximate nearest-neighbour search for hard negative mining. This environment used FAISS 1.14.1, compiled from source to enable GPU acceleration on the Hopper microarchitecture (compute capability sm_90) of the H200 GPUs. Pre-built FAISS wheels do not include sm_90 kernels, so a source build with the appropriate CUDA architecture flag was required to obtain full GPU acceleration for the inner-product nearest-neighbour searches that underpin negative candidate retrieval.

Distributed training was performed using PyTorch Distributed Data Parallel (DDP) across all eight GPUs within the single node. DDP was preferred over parameter-sharding alternatives such as FSDP because the RuModernBERT-base architecture fits comfortably within the VRAM of a single H200 device. Under this constraint, DDP provides near-linear throughput scaling without the additional synchronization overhead incurred by inter-device parameter communication. Experiment tracking, hyperparameter logging, metric curves, and artifact versioning across all runs were managed using the ClearML platform.

4.2 Training Data

The training corpus was assembled from the sources listed in Table 4.1. It is predominantly Russian, with multilingual and cross-lingual components included where they were methodologically relevant. The goal was to expose the reranker to a range of query formulations and document genres rather than a single dataset's distribution.

TABLE 4.1
Source datasets used in the construction of the final training corpus.

Dataset	Short description
9111_questions	Legal-domain question–passage pairs.
habr_qna	Technical community question–answer data collected from the Habr platform.
habr	Technical articles and associated textual content from the Habr community.
nq	Natural Questions, an open-domain question answering dataset built from real user queries and Wikipedia evidence passages [14].
ru_sci_bench	Scientific-domain retrieval data derived from academic literature.
swim_ir	Synthetic Wikipedia-based multilingual retrieval data designed for monolingual and cross-lingual dense retrieval across many languages [15].
mmarco	Russian subset of mMARCO, a multilingual version of the MS MARCO passage ranking dataset [47].
ru_news	News-domain retrieval pairs constructed from Russian-language news articles.
ru_stackoverflow	Programming and software engineering discussions collected from Stack Overflow in Russian.
yandex_q	General knowledge community questions and associated answer passages.

4.2.1 Source Datasets

These sources were unified and cleaned (§3.2.1), partitioned into training and validation splits (§3.2.2), then subjected to consistency filtering (§3.2.3) and hard negative mining (§3.2.4) as described in the subsections below.

The source datasets differ substantially in passage length, as shown in Figure 4.1: some corpora are concentrated almost entirely in the shortest bucket, whereas others contain a much larger share of long passages. This heterogeneity motivates the dataset-aware batching strategy described in §3.3.2, which keeps each

mini-batch within a single source to reduce padding overhead.

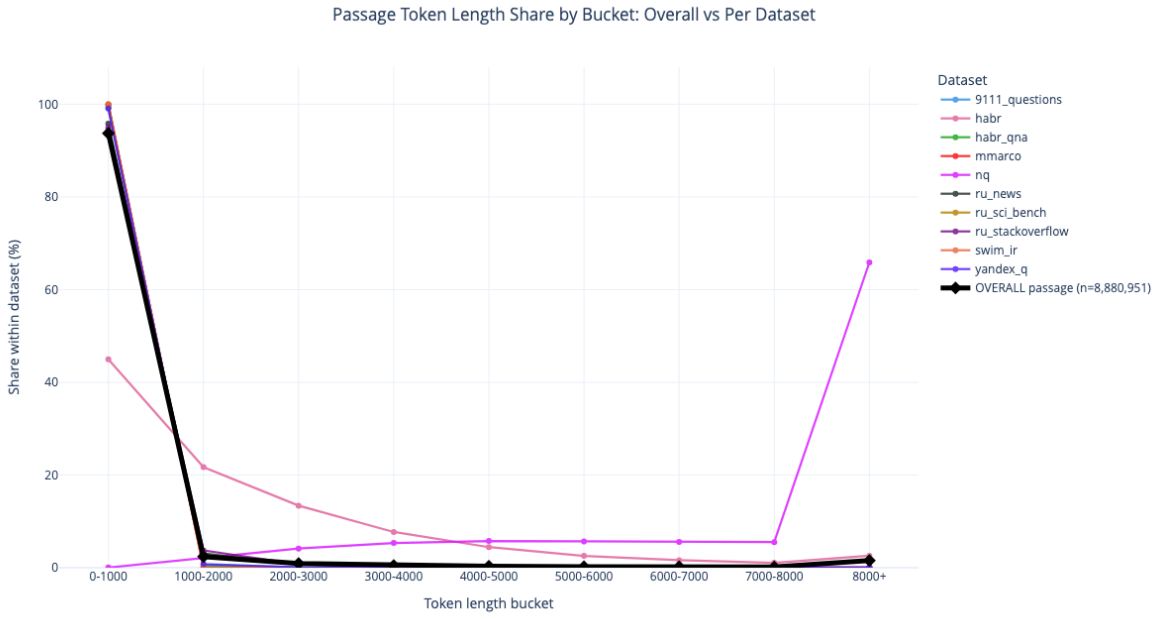


Fig. 4.1. Passage token-length distribution by dataset compared with the global passage distribution. Lengths are computed with the RuModernBERT-base tokenizer. The figure reports passage lengths only, because query lengths are substantially more homogeneous across datasets.

4.2.2 Consistency Filtering

Prior to training, all query–passage pairs in the training partition were subjected to the consistency-filtering procedure described in Chapter 3. A pair was retained only if the labeled positive passage appeared within the top-k results when the query was issued against the full passage index using Qwen3-Embeddings-0.6b as the retrieval model. For queries with a single positive passage, the threshold was set to $k = 30$. For queries with multiple positive passages, a hole-fitting strategy was applied: all other positives were temporarily removed from the candidate list when computing the rank of the target positive, and the threshold was relaxed to $k = 50$ to account for the increased competition among correct answers in dense

positive regions.

Three filtering strategies are compared in this work: no filtering (V1), filtering at $k{=}30$ (V2), and stricter filtering at $k{=}5$ (V3). The $k{=}30$ threshold allows positive passages ranked up to position 30 by $\mathcal{E}$; $k{=}5$ retains only pairs where the positive is among the top 5 retrieved results, selecting for the most unambiguous relevance signals at the cost of greater attrition.

Table 4.2 reports the pair counts under each strategy for each source corpus. The impact of filtering varied substantially across corpora: $9111_\mathrm{questions}$ exhibited the highest attrition under both thresholds, indicating that a large proportion of its query–passage pairs are not recoverable near the top of the retrieval ranking and are therefore likely to constitute noisy or weakly aligned annotations. $\mathrm{habr_qna}$ and $\mathrm{swim_ir}$ also showed high attrition, consistent with their paraphrastic and synthetic construction respectively. In contrast, nq and $\mathrm{ru_news}$ retained the majority of pairs under both thresholds.

TABLE 4.2
Training corpus size under three consistency-filtering strategies. V1: no filtering; V2: $k{=}30$; V3: $k{=}5$.

Dataset	**V1 (unfiltered)**	**V2 ($k{=}30$)**	**V3 ($k{=}5$)**
9111q	806,838	143,225	73,270
habr	271,653	247,138	216,421
habr-qna	267,786	90,475	47,333
mmarco	1,065,502	931,910	735,481
nq	221,247	214,690	176,985
ru-news	3,721,575	3,570,219	3,104,976
ru-sci-bench	326,172	335,932	310,178
ru-so	184,162	132,528	94,383
swim-ir	814,439	524,874	373,276
yandex-q	597,822	379,644	260,003
Total	**8,277,196**	**6,570,635**	**5,392,306**

4.2.3 Hard Negative Mining

Hard negatives were mined using $\mathrm{Qwen3\text{-}Embeddings\text{-}0.6b}$ as the dense bi-encoder retrieval model. This model produces dense vector representations of queries and passages; candidate negatives for each query were retrieved via inner-product nearest-neighbour search implemented in FAISS with GPU acceleration. An initial search depth of $k_0 = 128$ candidates was used; if the margin-filtered pool yielded fewer than the required number of valid negatives, the depth was widened iteratively up to a maximum of $k_{\max} = 2{,}048$ candidates, ensuring sufficient coverage even for queries in dense regions of the embedding space. Up to 15 hard-negative candidates were mined per query; queries for which at least $N{=}15$ valid candidates could not be identified even at the maximum search depth were discarded. Table 4.3 reports dropout for each corpus variant. For the unfiltered corpus (V1), dropout is substantial for several datasets — reaching 72.8% for 9111q and 13.6% overall — because many queries lack enough near-threshold passages in the embedding space. After consistency filtering, dropout drops to near zero: V2 ($k{=}30$) loses only 413 queries in total (0.01%), and V3 ($k{=}5$) loses none. This is a direct consequence of filtering: pairs retained by the consistency criterion are those whose positive passage is already well-represented in $\mathcal{E}$'s embedding space, which in turn ensures a dense neighbourhood of hard negative candidates. For the validation partition, which is always mined in its unfiltered form, 9111q retains the highest dropout at 38.6%; per-dataset figures are reported in Table 4.4.

Negative candidates were filtered according to a margin-based criterion: a candidate passage was accepted as a hard negative if its similarity score to the query fell within 5% of the mean similarity between the query and its labeled positive passages. This relative formulation — as opposed to a global similarity threshold — ensures that the hardness criterion adapts to the score distribution of each individual

TABLE 4.3
Query dropout due to insufficient hard negatives (N=15), training partition. V1: unfiltered; V2: k=30; V3: k=5. habr-qna has no V1 log.

	V1		V2 (k=30)		V3 (k=5)	
Dataset	**Queries**	**Drop%**	**Queries**	**Drop%**	**Queries**	**Drop%**
9111q	825,467	72.77	136,755	0.00	69,944	0.00
habr	288,718	5.44	246,621	0.00	216,117	0.00
habr-qna	–	–	90,387	0.00	47,319	0.00
mmarco	1,005,878	3.68	886,885	0.00	711,493	0.00
nq	235,990	2.89	214,690	0.00	176,985	0.00
ru-news	3,891,742	5.70	3,505,966	0.01	3,062,084	0.00
ru-sci-bench	361,438	2.10	335,932	0.00	310,178	0.00
ru-so	196,425	11.73	132,524	0.02	94,380	0.00
swim-ir	854,189	13.77	523,129	0.00	372,804	0.00
yandex-q	633,215	15.94	378,016	0.00	259,284	0.00
Total	**8,293,062**	**13.64**	**6,450,905**	**0.01**	**5,320,588**	**0.00**

query, preventing the systematic exclusion of hard negatives for queries whose positive passages occupy lower-similarity regions of the embedding space. All known positive passages, including cross-lingual variants of the same passage, were excluded from the candidate pool prior to negative selection to preserve label integrity.

The notion of *hardness* is defined relative to the Qwen3-Embeddings-0.6b retrieval model: passages that are confusable under this specific embedding function are designated as hard negatives. Consequently, biases in the bi-encoder's representation space may propagate into the training signal of the cross-encoder, a limitation shared with most hard-negative mining pipelines [59], [61].

4.2.4 Final Corpus Composition

Table 4.5 reports the final training set sizes produced by the corpus-assembly script for each of the three corpus variants, together with the single-variant validation set. The script assigns N=8 hard negatives per query (selected from the 15 mined

TABLE 4.4
Query dropout due to insufficient hard negatives (N=15), validation partition (mined in unfiltered form).

Dataset	Queries	Drop%
9111q	34,392	38.60
habr	12,046	1.10
habr-qna	11,787	13.98
mmarco	13,960	0.28
nq	9,833	0.19
ru-news	162,217	1.25
ru-sci-bench	14,496	0.21
ru-so	8,185	1.03
swim-ir	35,692	1.65
yandex-q	26,402	1.69
Total	**328,010**	**5.11**

candidates after positive exclusion) and applies all cross-lingual permutations ($\mathrm{permutation_mode=all}$), so bilingual corpora (mmarco, ru-sci-bench, swim-ir) yield roughly twice as many training rows as source query–passage pairs.

TABLE 4.5
Final corpus sizes after the assembly script (N=8 hard negatives per row, cross-lingual permutations applied). V1: unfiltered; V2: k=30; V3: k=5. The validation set is built from the unfiltered cleaned corpus (single variant).

Dataset	Split	Lang.	V1 pairs	V2 pairs	V3 pairs
mmarco	Train	EN+RU	2,053,928	1,792,870	1,347,544
9111q	Train	RU	225,105	136,799	69,961
habr-qna	Train	RU	156,979	90,475	47,333
nq	Train	EN	229,180	214,690	176,985
ru-news	Train	RU	3,743,531	3,569,832	3,104,976
ru-sci-bench	Train	EN+RU	694,754	648,478	588,906
ru-so	Train	RU	173,440	132,508	94,383
swim-ir	Train	EN+RU	745,219	524,865	373,271
yandex-q	Train	RU	536,046	379,591	259,967
habr	Train	RU	273,903	247,138	216,421
Total	Train		**8,832,085**	**7,737,246**	**6,279,747**
mmarco	Val	EN+RU			29,668
9111q	Val	RU			21,155
habr-qna	Val	RU			10,221
nq	Val	EN			9,814
ru-news	Val	RU			163,348
ru-sci-bench	Val	EN+RU			29,496
ru-so	Val	RU			8,101
swim-ir	Val	EN+RU			35,502
yandex-q	Val	RU			26,122
habr	Val	RU			11,941
Total	Val				**345,368**

Cross-lingual permutations expand bilingual corpora (EN+RU) to approximately twice the query count. For monolingual corpora, final counts may be slightly below input counts due to queries discarded for insufficient hard negatives after positive exclusion; per-dataset dropout figures appear in Table 4.3.

To better characterize the semantic structure of the corpus, the source datasets were grouped into broader thematic categories. Specifically, mmarco, yandex_q, swim-ir, and nq were treated as general knowledge and open-domain question answering resources; habr, habr_qna, and ru_stackoverflow were grouped as technical and IT-oriented data; ru_news, ru_sci_bench, and 9111_questions were retained as separate news, scientific, and legal categories, respectively. The thematic distribution for the V3 (k=5) filtered corpus is shown in Figure 4.2.

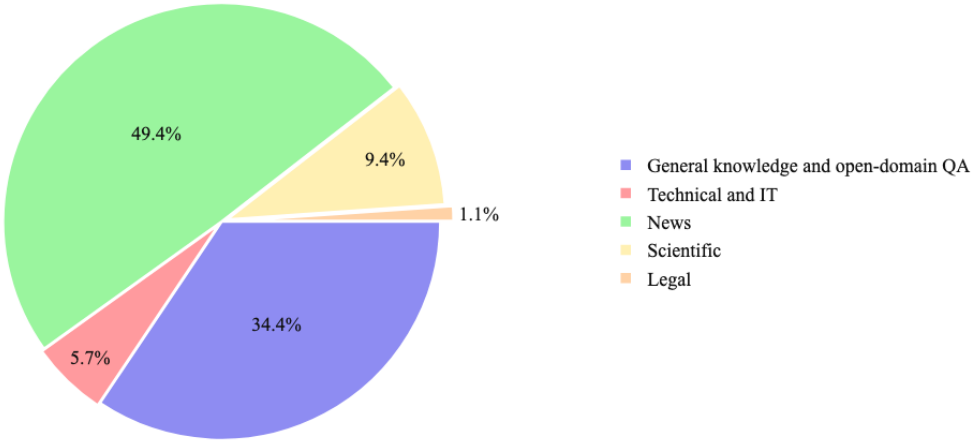


Fig. 4.2. Thematic distribution of the V3 (k=5) filtered training corpus by source category (6,279,747 total pairs).

The thematic composition is not uniform. In the V3 corpus, news data account for 49.4% of all training pairs because ru-news contributes a large natural volume of retrieval pairs. General-domain and open-domain QA sources contribute a further 34.4%, while scientific, technical, and legal domains each represent less than 10%. This imbalance matters for interpreting the results: the reranker is exposed more heavily to news-style passages than to legal or scientific documents. The relative weight of each category shifts modestly across corpus variants — relaxing the consistency filter (V2, V1) admits more pairs from sources such as

9111q, habr-qna, and swim-ir that suffer high attrition under strict filtering — but news and general-domain data dominate across all variants. The smaller domains are retained because the evaluation also covers them.

4.3 Training Configuration

4.3.1 Model and Hyperparameters

All experiments fine-tune RuModernBERT-base under a shared configuration to ensure comparability across ablation conditions. The per-device training batch size was set to 256, with a per-device evaluation batch size of 64. A mini-batch size of 128 was used in conjunction with gradient caching [65], which partitions the full per-device batch into sub-batches for the forward pass and accumulates exact gradients before the optimizer step. This reduces peak activation memory without changing the effective batch size: the loss still operates over the full batch, but only 128 pairs are materialised at once during the forward pass.

The maximum input sequence length was set to 512 tokens, the standard input length for BERT-family cross-encoder rerankers established by Nogueira and Cho [8] and adopted by the public sentence-transformers MS MARCO cross-encoder zoo [4]. RuModernBERT-base supports inputs up to 8,192 tokens, but transformer attention is $O(L^2)$, so doubling the input length roughly quadruples per-pair compute. The 512-token cap was retained to keep the full ablation grid (three filtering variants and four hard-negative conditions) within the available compute budget.

The loss function is a cached Multiple Negatives Ranking Loss with a temperature scale of 10.0, the default of the CrossEncoder.MultipleNegativesRankingLoss class in sentence-transformers v5.2.2.

The fused AdamW optimizer [74] was used with a learning rate of 1×10^{-5} and a weight decay of 0.01. A cosine learning-rate schedule with a warm-up ratio of 0.05 was applied: the first 5% of optimization steps ramped the learning rate linearly from zero to its peak value, after which it decayed following a cosine annealing curve. This warm-up strategy mitigates early-training instability caused by large gradient magnitudes when model parameters are far from the fine-tuning optimum [72].

Training was conducted for 3 epochs. Three epochs is a deliberate over-provision: Nogueira and Cho [8] reported that BERT-based reranking on MS MARCO saturates after fewer than 2% of training pairs, with no further dev-set improvement when training was extended by an additional 50M pairs. Validation was performed every 189 steps, yielding approximately 20 evaluation points per epoch — fine enough to localise the saturation point for early stopping without inflating evaluation overhead. Model selection was based on validation MAP; the checkpoint attaining the highest MAP on the held-out partition was retained for final evaluation.

Mixed-precision training [75] using $\mathrm{bfloat16}$ was enabled throughout, combined with TF32 acceleration on the tensor cores of the H200 GPUs. Gradient checkpointing was not used. Gradient accumulation was not used. Gradient clipping was applied with a maximum gradient norm of 1.0. A fixed random seed ($\mathrm{seed}{=}12$) was set once prior to all experiments to ensure full reproducibility of weight initialization, data ordering, and sampling; it was not reselected at any point.

Hyperparameters not tuned. The learning rate, weight decay, warm-up ratio, schedule shape, AdamW configuration, and MNRL temperature were not subject to a hyperparameter sweep. Each was set to a conservative value within the range

commonly used for BERT-family fine-tuning and held fixed across all ablation conditions. The compute budget was deliberately allocated to the data-side ablations (consistency filtering, hard-negative composition, SLERP) hypothesised to dominate the outcome for a Russian-specialised cross-encoder. The cost of this choice is that the absolute scores reported in Chapter 5 should be read as a single point in optimiser-hyperparameter space; the relative comparisons between ablation conditions remain valid because all conditions share that point.

4.4 Evaluation Protocol

The evaluation protocol is designed to provide a contamination-aware assessment of model performance on held-out Russian-language retrieval benchmarks. Benchmark contamination — the inadvertent exposure of a model to test data during training — can significantly inflate evaluation scores in information retrieval systems [76]. This issue is particularly pronounced for reranking models trained on mixtures of widely reused datasets. The protocol described here addresses contamination through dataset selection and lineage analysis rather than post-hoc deduplication.

4.4.1 Dataset Selection

Evaluation datasets are drawn from the RusBEIR benchmark suite [10], a Russian-language IR collection modelled on BEIR [31]. The candidate pool is pre-filtered to datasets whose task type falls into either *Information Retrieval* (IR) or *Question Answering* (QA), the two task families most directly relevant to passage reranking. Argument retrieval, duplicate-question retrieval, and citation-prediction datasets are excluded from the evaluation mixture on the grounds that these task

formulations differ structurally from the retrieval scenarios the reranker is designed to serve.

4.4.2 Contamination-Control Protocol

The contamination-control procedure is illustrated in Figure 4.3 and consists of three stages:

- **Dataset lineage analysis:** each candidate dataset is analyzed with respect to its construction methodology, the original source of its passage corpus, and its query generation procedure. For translated datasets, the source of the original English corpus is traced. For natively Russian datasets, the primary publication and data pipeline are consulted.
- **Principled exclusion:** any dataset whose passage corpus directly overlaps with a training source is excluded from evaluation. Two datasets were excluded on this basis (§4.4.3). Datasets exhibiting only corpus-level overlap — where the underlying text collection is shared but queries and relevance annotations are independently constructed — are retained with explicit documentation of the contamination risk (Appendix A).
- **Equal-weight aggregation:** the nine remaining datasets receive equal weight in the aggregate score, consistent with the evaluation design described in §3.4.3.

4.4.3 Evaluation Mixture

Table 4.6 lists all eleven candidate datasets, their task types, passage sources, contamination risk assessment, and final weights.

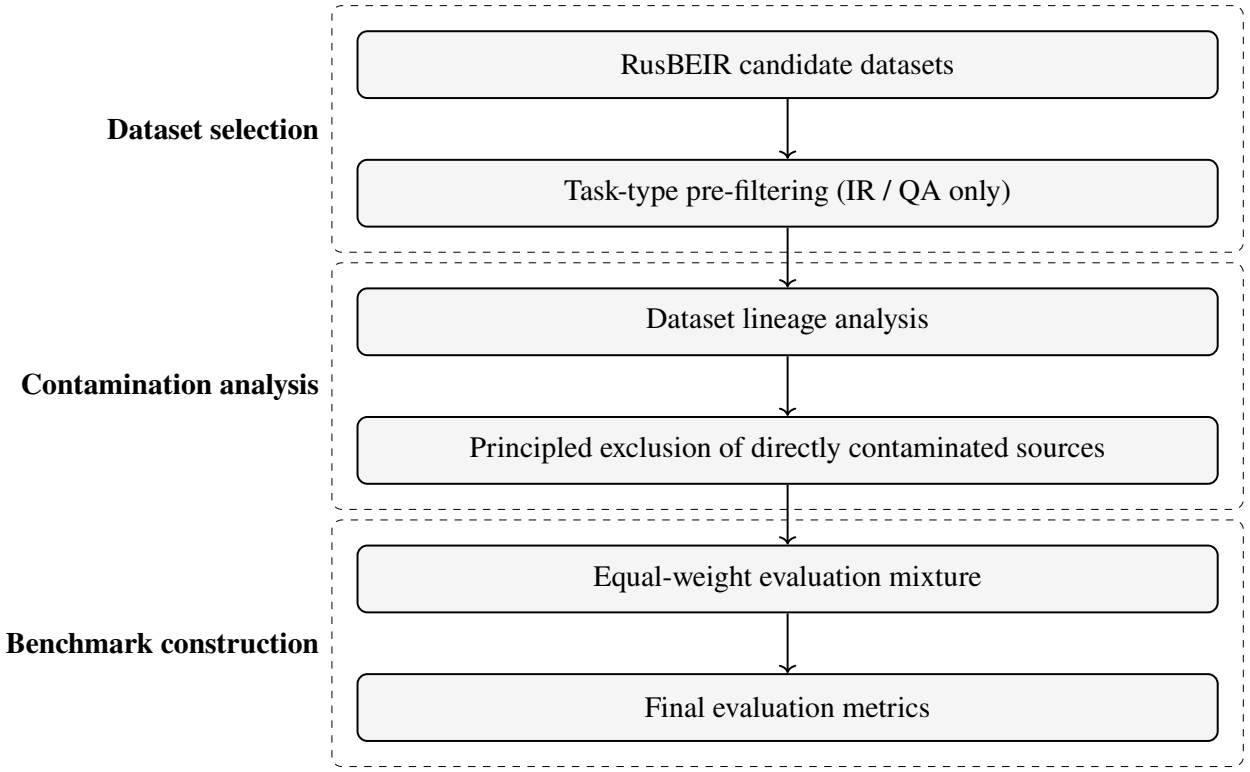


Fig. 4.3. Contamination-aware evaluation pipeline for constructing the final benchmark.

ruscibench-retrieval is excluded because its passage corpus is drawn from the same ruSciBench collection used as a training source (ru_sci_bench), constituting direct passage-level contamination. rus-scidocs is excluded due to high corpus-family overlap with the same training source.

Three of the nine active datasets (rubq, sberquad-retrieval, and wikifacts-sents) use Russian Wikipedia as their passage corpus. The swim_ir training dataset generates synthetic queries over Russian Wikipedia paragraphs, creating corpus-level overlap: the passage texts are drawn from the same underlying collection. Because the queries in these evaluation datasets originate from a structurally different process (Wikidata knowledge-graph questions, reading comprehension annotations, and factual probes respectively), the contamination is classified as corpus-level rather than pair-level and is documented rather than triggering exclusion. Per-dataset

TABLE 4.6
RusBEIR evaluation datasets. The nine active datasets (weight $= 1/9$) form the primary evaluation mixture; two are excluded due to direct passage-corpus overlap with training sources. Full per-dataset contamination analysis is in Appendix A.

Dataset	Task	Passage source	Contamination risk	Weight
rubq	Open-domain QA	Russian Wikipedia	Medium (corpus, swim_ir)	$1/9$
ria-news	News IR	RIA Novosti	None	$1/9$
rus-nfcorpus	Bio-Medical IR	PubMed (translated)	None	$1/9$
rus-trec-covid	Bio-Medical IR	CORD-19 (translated)	None	$1/9$
sberquad-ret.	Reading comp. IR	Russian Wikipedia	Medium (corpus, swim_ir)	$1/9$
wikifacts-sents	Fact retrieval	Russian Wikipedia	Medium (corpus, swim_ir)	$1/9$
rus-fiqa	Financial QA	StackExch. Fin. (transl.)	Low	$1/9$
rus-xquad	QA	Eng. Wikipedia (transl.)	Low–medium (corpus, nq)	$1/9$
rus-xquad-sents	QA (sent.-level)	Eng. Wikipedia (transl.)	Low–medium (corpus, nq)	$1/9$
ruscibench-ret.	Scientific IR	ruSciBench	**Direct** (ru_sci_bench)	0
rus-scidocs	Scientific IR	SciDocs (translated)	High (ru_sci_bench family)	0

contamination details are provided in Appendix A.

4.4.4 First-Stage Retrieval Setup

For each benchmark dataset, BM25 retrieves the top-100 candidate passages per query from the full benchmark corpus using the kaengreg offline dataset bundle. The reranker then scores each of the 100 query–passage pairs and produces a final ranked list. Evaluation metrics are computed over this reranked output. Fixing the first-stage retriever across all compared models ensures that observed metric differences are attributable solely to the reranker.

4.4.5 Evaluation Metrics

Primary benchmark metric is NDCG@10 (see §3.4.4). Secondary metrics reported are MAP@10 and MRR@10. All models are evaluated under identical conditions: the same BM25 candidate pool, the same offline dataset bundle, and

the same metric computation library, ensuring that performance differences reflect reranker quality rather than experimental artifacts.

4.4.6 Summary

The evaluation protocol selects nine RusBEIR datasets covering bio-medical IR, news retrieval, open-domain QA, financial QA, reading comprehension retrieval, fact retrieval, and cross-lingual QA. Two datasets are excluded due to direct passage-corpus overlap with training sources; the remaining contamination risks are documented in the appendix. All nine active datasets receive equal weight in the aggregate NDCG@10 score. A fixed BM25 first stage isolates the reranker's contribution across all comparisons.

4.5 Ablation: Consistency Filtering

4.5.1 Ablation Design

To quantify the contribution of the consistency-filtering stage, an ablation study was conducted comparing the following dataset formation variants, with the model architecture and all training hyperparameters held fixed:

V1 — Unfiltered: The full training split, comprising 8,277,196 pairs, without consistency filtering. This variant establishes the baseline attainable when label noise is not addressed.

V2 — Filtered (k=30): The training split after applying the consistency criterion with threshold k=30, comprising 6,570,635 pairs. This is the primary training configuration used throughout this study.

V3 — Strictly filtered (k=5): The training split after applying the stricter consistency criterion with threshold k=5, comprising 5,392,306 pairs. This variant retains only pairs for which the positive passage is ranked in the top 5 by $\mathcal{E}$, selecting for high-confidence relevance signals at the cost of a 34.8% reduction from the unfiltered pool.

4.5.2 Results

All three variants were trained under the shared configuration of Section 4.3.1 with the 8HN+2RN negative sampling scheme, so the threshold of the consistency filter is the only difference between runs. The same early-stopping criterion was applied to each run; because the criterion triggers at a different point in each trajectory, the runs terminate at different steps. Every saved checkpoint was evaluated on the nine active RusBEIR datasets (Section 4.4.3) using the BM25 top-100 first stage of Section 4.4.4.

Figure 4.4 tracks the six benchmark metrics across training for the three runs, with bge-reranker-v2-m3 drawn as a horizontal reference. Checkpoints before step 1,000 are omitted: all runs behave alike in that region, and the early values add little. The curves share one shape. Each metric climbs steeply through the first few thousand steps, then settles onto a plateau that the unfiltered (V1) and k=30 (V2) runs reach at a visibly higher level than the strict k=5 run (V3). V1 and V2 stay within checkpoint-to-checkpoint noise of each other for the entire trajectory.

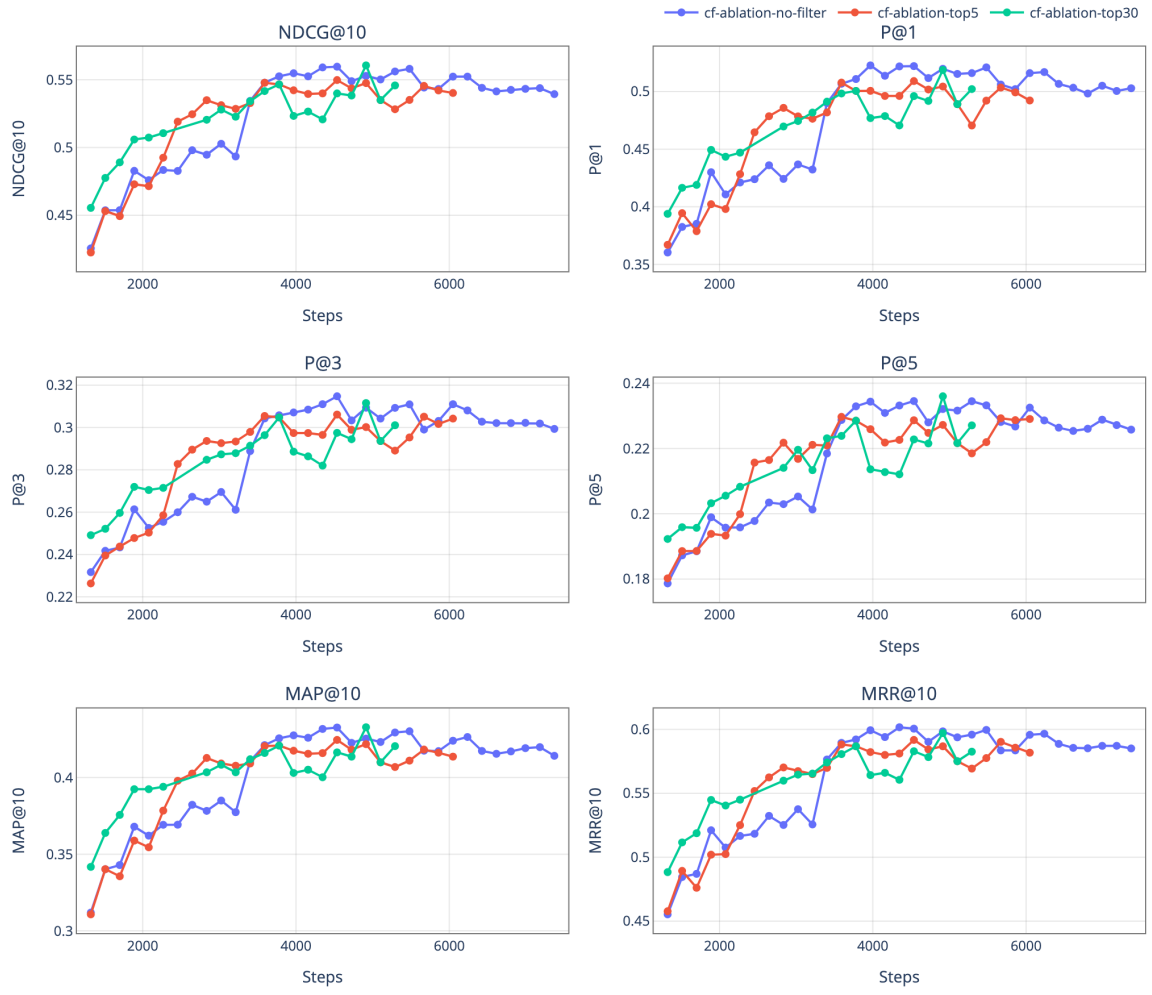


Fig. 4.4. Benchmark metrics versus training step for the three consistency-filtering variants, aggregated over the nine active RusBEIR datasets (equal weight). The dashed line marks bge-reranker-v2-m3. Checkpoints before step 1,000 are omitted.

Table 4.7 reports the best checkpoint of each run, selected by benchmark NDCG@10. The neighbouring checkpoints of V1 and V3 sit within a few thousandths of these peaks, so their figures describe a stable plateau; V2 peaks more sharply at step 4,914, and its value is better read together with the curve than as a stable point estimate.

TABLE 4.7
Impact of consistency filtering on benchmark quality. Best checkpoint per run (by NDCG@10), aggregated over the nine active RusBEIR datasets. bge-reranker-v2-m3 is an off-the-shelf reference, not part of the ablation. Bold marks the best value among the three variants per column.

Variant	Step	NDCG@10	MAP@10	MRR@10	P@1
V1 — Unfiltered	4,536	0.560	0.433	**0.601**	**0.522**
V2 — Filtered (k=30)	4,914	**0.561**	**0.433**	0.597	0.519
V3 — Filtered (k=5)	4,536	0.550	0.425	0.592	0.509
bge-reranker-v2-m3 *(reference)*	—	0.587	0.453	0.630	0.554

4.5.3 Analysis

The ablation asks whether discarding pairs whose labeled positive is not recoverable near the top of the retrieval ranking produces a better reranker. The results give a split answer: filtering at k=30 neither helps nor hurts, while the stricter k=5 filter hurts.

V1 and V2 converge to the same benchmark level. Their peak NDCG@10 differs by 0.001 (0.560 against 0.561) and their MAP@10 is identical to three decimals, with V1 holding a slight edge on MRR@10 and P@1. Removing 20.6% of the training pairs (1.7M of 8.3M) under the k=30 criterion therefore leaves final quality unchanged. The reading is that the pairs this filter removes do not inject enough contradictory gradient signal to degrade the model, and removing them does not concentrate the remaining signal enough to help it. With eight hard negatives per query, the MNRL objective absorbs the label noise present in the unfiltered corpus at this data scale.

V3 is the outlier. The k=5 filter drops 34.8% of the pairs and costs about one point of NDCG@10 (0.550 against 0.561 for V2), with a matching shortfall on MAP@10 and MRR@10. The loss appears at every metric and at every point on the plateau, so it is a property of the run rather than an unlucky checkpoint.

The likely cause is over-filtering: many positives ranked between position 5 and position 30 by $\mathcal{E}$ are genuinely relevant, and excluding them shrinks the corpus past the point where cleaner labels can compensate. Once the filter is this aggressive, training volume matters more than the marginal reduction in noise.

The ordering of the three variants is stable, but none reaches bge-reranker-v2-m3, which leads by 3 to 4 points of NDCG@10 (0.587 against 0.561 for V2). This gap is expected and is not the target of the ablation. The reference is a larger, separately trained reranker, whereas these runs fine-tune a 150M-parameter RuModernBERT-base and differ from each other only in the filtering threshold. The experiment isolates the effect of that threshold; it does not attempt to close the distance to the reference model.

One caveat qualifies the null result for k=30. The pairs removed by filtering are concentrated in 9111q (legal, 81.6% attrition) and habr-qna (technical, 66.9% attrition), as Table 4.2 shows. The evaluation mixture contains no legal dataset and only indirect technical coverage, so the benchmark barely probes the domains where filtering reshapes the training distribution the most. The finding that k=30 filtering is cost-free should be read with that limitation in mind: it holds for this benchmark, and a legal- or technical-heavy evaluation might reveal a difference that the present mixture cannot.

4.6 Ablation: Hard Negative Sampling

4.6.1 Ablation Design

To quantify the contribution of hard negatives to reranking quality, an ablation study was conducted over three negative-composition conditions on the V2 (k=30) training corpus introduced in the consistency-filtering ablation of §4.5, with the

model architecture and all training hyperparameters held fixed:

0HN+2RN — random-only baseline: No mined hard negatives are used; the contrastive loss sees only $K_R{=}2$ random in-batch negatives per query. This condition isolates the contribution of in-batch random negatives alone and tests the hypothesis (Ablation C of §3.4.5) that random negatives are insufficient for high-precision reranking.

8HN+2RN — mixed strategy: $K_H{=}8$ mined hard negatives are concatenated with $K_R{=}2$ random in-batch negatives. This is the configuration used in the consistency-filtering ablation (§4.5) and in the headline comparison of Chapter 5.

15HN+2RN — maximum hard-negative load: All $K_H{=}15$ mined candidates are used together with $K_R{=}2$ random negatives. This condition saturates the hard-negative budget and tests whether additional hard negatives provide further discriminative signal or instead amplify false-negative contamination.

All three conditions share the same training data, the same $\mathrm{Qwen3\text{-}Embeddings\text{-}0.6b}$-mined negative pool, and the same optimiser configuration of §4.3.1. The only varied quantity is the count of mined hard negatives included in the MNRL loss. As in §4.5, every saved checkpoint was evaluated on the nine active rusBEIR datasets (§4.4.3) under the BM25 top-100 first stage of §4.4.4.

4.6.2 Results

Figure 4.5 traces the six benchmark metrics across training for the three conditions. Two findings are visible at once. First, the random-only baseline (0HN+2RN) sits clearly below the two mixed conditions on every metric and at

every point of the trajectory: it plateaus around NDCG@10 ≈ 0.45 while the mixed conditions converge near 0.56. Second, the 8HN+2RN and 15HN+2RN trajectories are visually indistinguishable across the full training run — on NDCG@10, MAP@10, MRR@10, and the three precision-at-cutoff metrics they overlap within checkpoint-to-checkpoint noise.



Fig. 4.5. Benchmark metrics versus training step for the three hard-negative sampling conditions (0HN+2RN, 8HN+2RN, 15HN+2RN), aggregated over the nine active rusBEIR datasets (equal weight). All three runs share the V2 (k=30) training corpus and the shared hyperparameters of §4.3.1.

The 0HN+2RN trajectory is also visibly noisier than the other two: it

oscillates substantially across consecutive checkpoints, whereas the 8HN+2RN and 15HN+2RN curves rise smoothly and stabilise after roughly step 3,000.

4.6.3 Analysis

The ablation gives a clean answer to two of the three sub-questions raised in §3.4.5: hard negatives are necessary, and beyond a moderate count they are not additive.

The first conclusion follows directly from the 0HN+2RN baseline. Random in-batch negatives alone yield NDCG@10 approximately ten percentage points below the mixed conditions. In-batch random negatives are almost always topically unrelated to the query — the dataset-aware batch sampler (§3.3.2) draws all in-batch pairs from a single source dataset, but within that source the positive passages of unrelated queries do not concentrate near the current query in the cross-encoder's score space. The MNRL objective therefore reduces to separating the positive from a set of trivially distinguishable distractors, which is a much weaker discriminative task than the reranker is asked to perform at evaluation time, where BM25 has already filtered the candidate pool to lexically plausible passages. The noisy training trajectory of the 0HN+2RN run is consistent with this reading: with no hard signal in the batch, the gradient direction at each step is dominated by whatever weak distractors happen to be sampled, producing higher checkpoint-to-checkpoint variance.

The second conclusion follows from the overlap of the 8HN+2RN and 15HN+2RN curves. Increasing the per-query hard-negative count from 8 to 15 nearly doubles the discriminative load on the loss but yields no measurable improvement on any of the six metrics. Two interpretations are compatible with this null result. The first is a saturation argument: once a small number of genuinely

hard negatives are present in each contrastive batch, the gradient signal needed to sharpen the decision boundary is already provided, and adding more similarly hard examples produces redundant rather than complementary information. The second is a false-negative-contamination argument: the pool of 15 mined candidates per query has a higher base rate of unlabelled-positive passages than the top-8 subset, and the extra discriminative pressure on those false hard negatives partially cancels the benefit of the additional true hard negatives. The two mechanisms cannot be separated by the experiments reported here, but both predict the observed plateau.

A practical consequence is that the 8HN+2RN configuration is preferred over 15HN+2RN. It achieves equivalent benchmark quality while reducing per-step compute and memory: each training pair materialises 8 fewer cross-encoder forward passes through the backbone, which under the gradient-caching schedule of §3.3.3 translates directly into a smaller effective mini-batch budget. The 8HN+2RN run is therefore retained as the headline configuration in Chapter 5.

One caveat tempers the generality of this finding. The hard negatives used here are mined under a single retrieval model $\mathcal{E}$, and the saturation observed at $K_H{=}8$ may reflect a ceiling imposed by $\mathcal{E}$'s representational diversity rather than a property of the cross-encoder. A stronger or differently-trained retrieval model could surface harder candidates within its top-15, in which case the 15HN+2RN regime might re-separate from 8HN+2RN. Exploring this interaction is left to future work and is discussed alongside the related retrieval-model sensitivity question in §5.4.

4.7 Ablation: SLERP

4.7.1 Ablation Design

Spherical Linear Interpolation (SLERP) merges two weight vectors along the geodesic of the unit hypersphere they span, preserving parameter norms and avoiding the destructive interference that naive weight averaging can introduce when the merged checkpoints lie far from a shared low-loss basin [44]. The $\mathrm{multi_slerp}$ method generalizes this construction to more than two checkpoints in a single step by interpolating jointly on the hypersphere rather than chaining pairwise SLERP, which avoids the order-dependence and accumulated norm drift of the sequential formulation. The interest in checkpoint merging for cross-encoder training is practical: fine-tuning runs are stochastic, the best validation checkpoint is sensitive to the evaluation interval, and the cost of a merge is negligible compared to retraining.

Two experiments are conducted to characterize the regimes in which SLERP is and is not useful for this pipeline:

Experiment 1 — across training runs. The best-by-NDCG@10 checkpoints of the V1, V2, and V3 consistency-filtering runs (Section 4.5) are merged. Each is the endpoint of a different optimization trajectory, so the three weight vectors are not guaranteed to sit in a shared basin. This experiment asks whether SLERP can combine models that were trained on *different data* into a single reranker that inherits the strengths of each.

Experiment 2 — within a single training run. Five consecutive late-training checkpoints from the V2 run are merged. These checkpoints lie on a contiguous segment of the same optimization trajectory and therefore occupy

a much smaller region of weight space. This experiment asks whether SLERP across nearby optima can smooth out the checkpoint-to-checkpoint noise documented in Figure 4.4 and outperform the single best checkpoint of the run.

Both experiments use the same evaluation pipeline as the rest of Chapter 4: BM25 top-100 first stage, then reranking with the model under test. Reported metrics in this section are means across the eleven RusBEIR datasets, including the two otherwise excluded by the contamination protocol of Section 4.4.3, so that the SLERP comparison itself is not affected by which datasets are active in the aggregate; the relative ordering of models is the quantity of interest, and reranker-internal merging cannot introduce contamination that the individual checkpoints did not already have.

4.7.2 Results

Table 4.8 and Figure 4.6 report the merge across training runs. The merged model tracks the strongest input on NDCG@10 and MAP@10 to within rounding, takes the top value on MRR@10 and P@1, and avoids being pulled down by the weakest of the three inputs. The figure makes the overall picture explicit: the four bars per metric are visually indistinguishable except for the V3 input, which sits noticeably below the other three.

TABLE 4.8
SLERP across the best checkpoints of the three consistency-filtering runs (V1, V2, V3). Mean across the eleven RusBEIR datasets. Bold marks the best value per column.

Model	NDCG@10	P@1	P@3	P@5	MAP@10	MRR@10
V1 best	0.5010	0.4864	**0.2947**	0.2190	0.3847	0.5634
V2 best	**0.5027**	0.4836	0.2919	**0.2213**	**0.3854**	0.5612
V3 best	0.4892	0.4698	0.2837	0.2135	0.3728	0.5493
merged	0.5021	**0.4869**	**0.2947**	0.2203	0.3850	**0.5676**

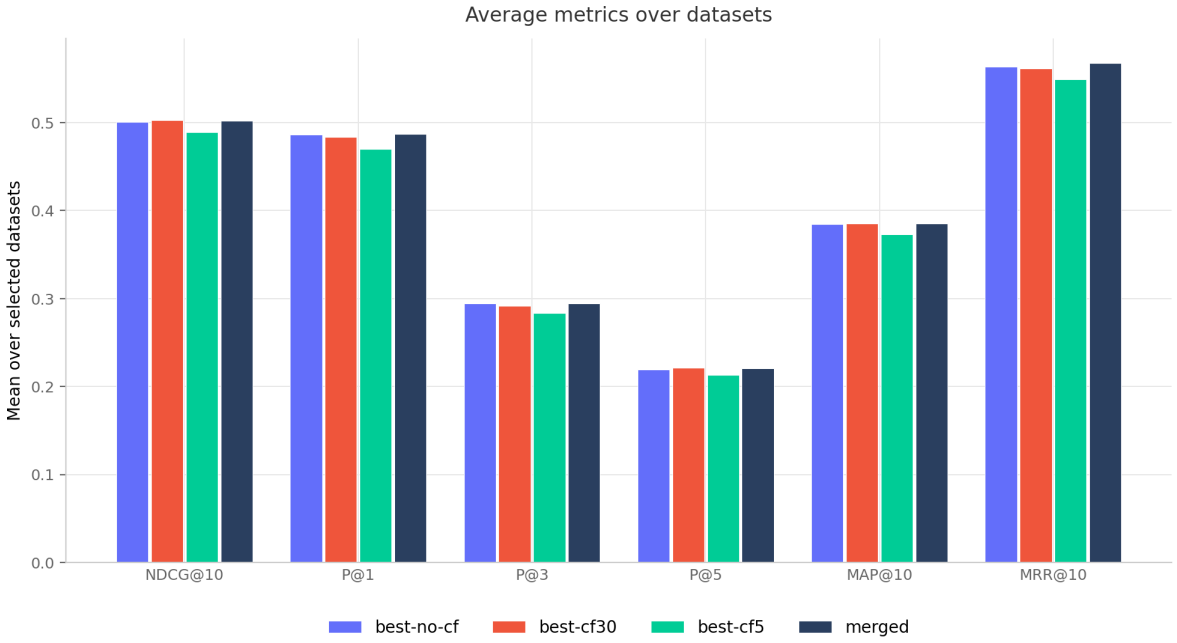


Fig. 4.6. Experiment 1: best checkpoints of the V1, V2, V3 runs versus their SLERP merge. Mean over the eleven RusBEIR datasets.

Table 4.9 and Figure 4.7 report the merge within the V2 run. On every reported metric the merged model sits below the single best checkpoint and above the mean of the five inputs. The figure shows that the five constituent checkpoints are themselves ordered on a clear upward trend across the late-training segment, and that the merged bar lands close to the second-best checkpoint rather than the best.

TABLE 4.9
SLERP across five consecutive checkpoints of the V2 (k=30) training run. Mean across the eleven RusBEIR datasets. Bold marks the best value per column.

Checkpoint	NDCG@10	P@1	P@3	P@5	MAP@10	MRR@10
step 4,158	0.4687	0.4406	0.2656	0.1981	0.3574	0.5264
step 4,347	0.4636	0.4324	0.2618	0.1970	0.3530	0.5208
step 4,536	0.4817	0.4572	0.2770	0.2078	0.3684	0.5434
step 4,725	0.4796	0.4528	0.2733	0.2060	0.3655	0.5379
step 4,914	**0.5027**	**0.4836**	**0.2919**	**0.2213**	**0.3854**	**0.5612**
merged	0.4980	0.4793	0.2893	0.2182	0.3816	0.5590

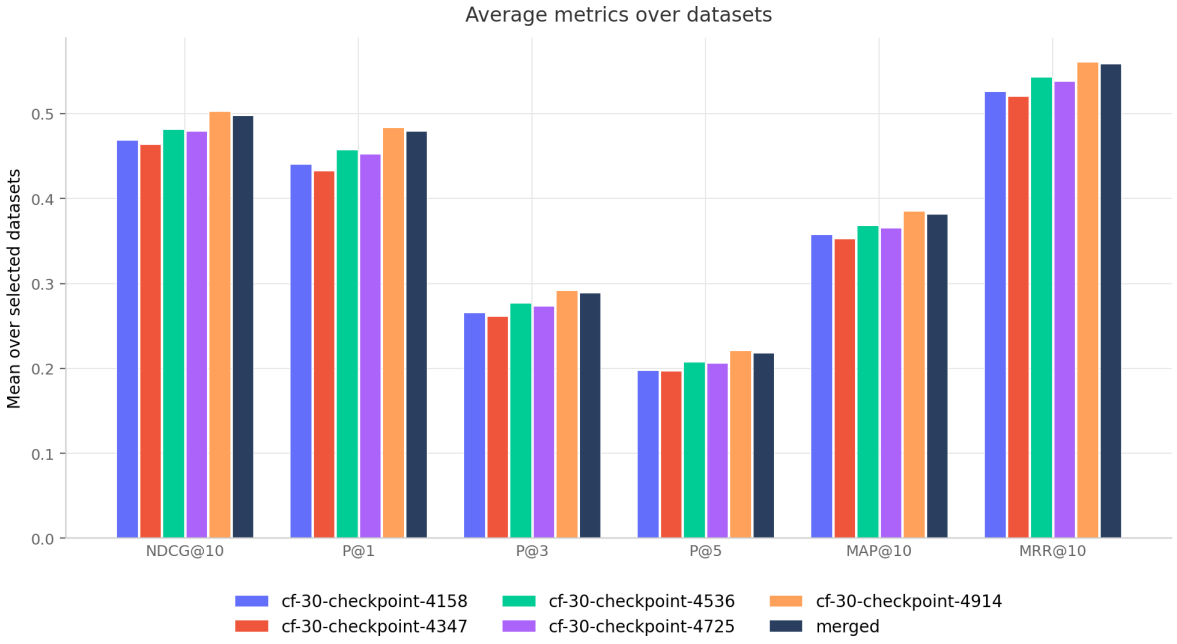


Fig. 4.7. Experiment 2: SLERP across five consecutive checkpoints from the V2 (k=30) run versus the merged model. Mean over the eleven RusBEIR datasets.

4.7.3 Analysis

Neither experiment delivers a clear improvement from SLERP. The across-runs merge ties the best input on the primary ranking metrics and only ekes out a fractional gain on early-rank measures, and the within-run merge actively underperforms the best constituent checkpoint on every metric reported.

In Experiment 1 the merge matches the strongest individual run on NDCG@10 and MAP@10 to within rounding. The small lead it takes on MRR@10 and P@1 is the same order of magnitude as the checkpoint-to-checkpoint noise visible in Figure 4.4, so it cannot be read as a real gain; at best it indicates that joint interpolation across runs trained on different data does not destructively interfere. The merge appears to land near the level of the best input rather than meaningfully above it.

Experiment 2 is the clearer negative result. The five checkpoints lie on a still-improving trajectory, and averaging the best one with weaker predecessors pulls the merged model toward the trajectory mean rather than the tip. The outcome is a uniform drop versus the single best checkpoint, which is the more honest baseline for that setting. Within an improving training run, picking the best checkpoint dominates merging.

Together the two experiments suggest that SLERP in the form applied here — joint $\mathrm{multi_slerp}$ over a small set of converged or late-training checkpoints — does not yield the clean robustness gain that the literature on checkpoint merging often reports. It is essentially a no-op when applied across runs that have already converged, and it is harmful when applied along an improving trajectory. The final model carried into Chapter 5 is therefore the best single checkpoint rather than a merged variant. A more favourable regime would likely require either a denser sweep of converged checkpoints from genuinely diverse training trajectories, or a different merging recipe than the one tested here.

Chapter 5

Evaluation and Discussion

5.1 Quantitative Evaluation

5.1.1 Experimental Setup

Following the ablations, the best-performing configuration in terms of NDCG@10 — 8 hard negatives and 2 random negatives (8HN+2RN) — was selected for the final model. All hyperparameters were kept identical to those described in Section 4.3.1, with the maximum sequence length increased to 4096 tokens, and the final model was retrained under this setting.

All models are evaluated on the nine-dataset rusBEIR macro-average (Section 4.4.3), excluding ruscibench-retrieval_dev and rus-scidocs_test due to English-query contamination.

Primary metric. NDCG@10 is adopted as the primary evaluation metric. Unlike MRR, which rewards only the rank of the first relevant result, NDCG@10 measures the quality of the entire top-10 list through a position-discounted gain formulation that accounts for graded relevance judgements. This makes it well suited for a

reranking setting, where the model reorders a fixed candidate set and the usefulness of the ranked output to a downstream generator depends on the quality of the full top-k context window. MAP@10 penalises for relevant documents absent from the BM25 candidate set (failures attributable to the first-stage retriever rather than the reranker) and is therefore a less direct measure of reranking quality specifically. NDCG@10 is also the headline metric on BEIR and MS MARCO leaderboards, ensuring comparability with published baselines. MAP@10, MRR@10, and P@3 are reported as supporting metrics.

5.1.2 Results

Table 5.1 reports macro-averaged NDCG@10, MAP@10, MRR@10, and P@3 over the nine active rusBEIR datasets for all evaluated models. Per-dataset breakdowns for each metric are provided in Appendix B.

TABLE 5.1
Comparison of retrieval models on the rusBEIR benchmark (macro-average over 9 datasets; ruscibench-retrieval_dev and rus-scidocs_test excluded due to English queries). The best score per column is **bolded**. Params denotes model size in millions of parameters. All scores are from a single training run; no averaging over multiple seeds was performed.

Model	Params (M)	NDCG@10	MAP@10	MRR@10	P@3
First-stage retrieval					
BM25	N/A	0.455	0.345	0.485	0.244
Cross-encoders					
Qwen3-Reranker-0.6B	596	0.545	0.410	0.588	0.307
bge-reranker-v2-m3	568	**0.587**	**0.453**	**0.630**	**0.333**
Fine-tuned cross-encoder (this work)					
RuModernBERT-base, 8HN+2RN	150	0.560	0.432	0.597	0.311

5.1.3 Analysis

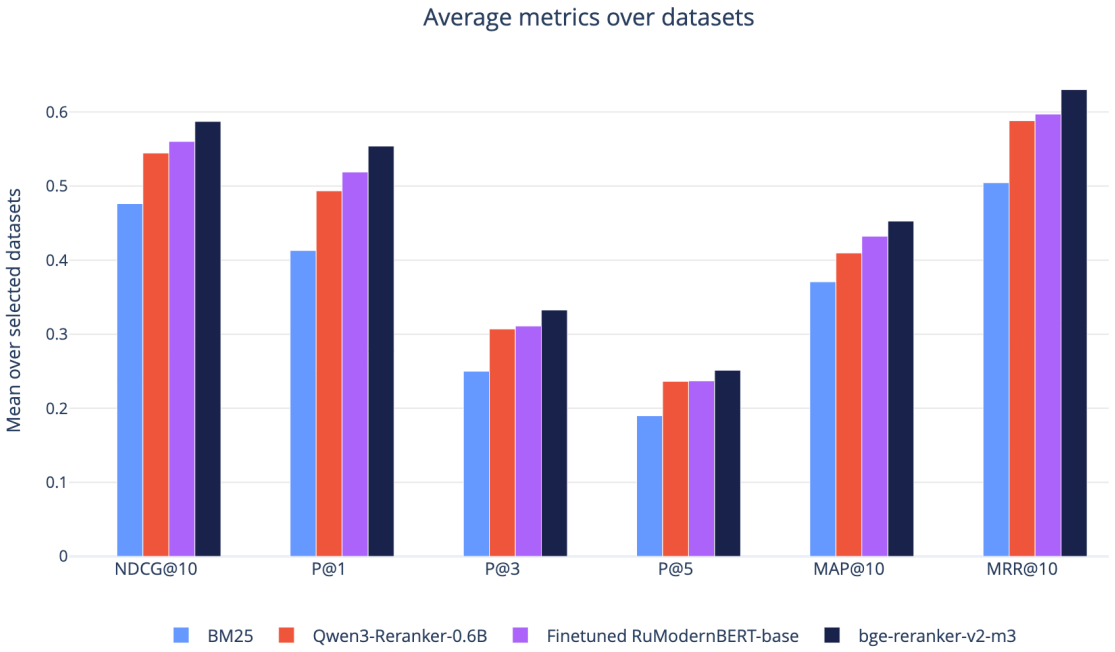


Fig. 5.1. Macro-averaged retrieval metrics over the nine active rusBEIR datasets for all evaluated models. Each group of bars corresponds to one metric; bar colour identifies the model (see legend). Exact values are reported in Table 5.1; per-dataset breakdowns are in Appendix B.

The results in Table 5.1 and Figure 5.1 reveal three performance tiers. BM25 establishes the first-stage retrieval ceiling (NDCG@10 = 0.455). bge-reranker-v2-m3 leads all models on every metric, with NDCG@10 = 0.587. The fine-tuned RuModernBERT-base (150 M) and Qwen3-Reranker-0.6B (596 M) form a middle tier, with our model outperforming Qwen3 on all four metrics.

Gap to BGE. bge-reranker-v2-m3 leads by 2.7 pp NDCG@10 over the next best model. The per-dataset breakdown (Appendix B) shows the gap is largest on news and biomedical retrieval, consistent with BGE having been trained on a substantially larger and more diverse multilingual retrieval corpus spanning many domains and query styles.

Training dynamics on the out-of-distribution test set. Figure 5.2 traces the macro-averaged NDCG@10 on the rusBEIR benchmark at every evaluation checkpoint throughout training. Because rusBEIR is never seen during fine-tuning, this curve directly measures out-of-distribution generalisation rather than in-distribution fit, making it a reliable indicator of whether ranking improvements are genuine or artefacts of training data memorisation. The model surpasses the BM25 ceiling within the first few hundred steps and crosses the $\mathrm{Qwen3\text{-}Reranker\text{-}0.6B}$ reference line around checkpoint 2 500, after which it remains consistently above it for the rest of training. The curve is non-monotone: it oscillates around a rising trend in the early phase and stabilises from approximately checkpoint 4 000 onward. This pattern is characteristic of a model adapting to a distribution shift; the training signal drives ranking improvement on seen data, but the test-set curve lags and fluctuates until representations generalise. The peak is reached at checkpoint 4914 (NDCG@10 = 0.560), after which the model shows marginal degradation, suggesting mild overfitting to the fine-tuning distribution in later steps. The $\mathrm{bge\text{-}reranker\text{-}v2\text{-}m3}$ reference line remains above the trajectory throughout, confirming that the gap to BGE is not a training-length artefact but a structural difference attributable to the two models' pre-training and fine-tuning data.

In-training optimisation dynamics. Figure 5.3 reports the in-training signals — loss, gradient norm, and learning rate — across all optimisation steps of the same run. The training loss decreases steeply through the warm-up phase and then settles onto a slowly declining plateau, consistent with the rapid saturation expected of BERT-family cross-encoder fine-tuning. The learning-rate panel confirms the configured schedule of §4.3.1: a linear warm-up over the first 5% of steps to a peak of 1×10^{-5}, followed by cosine decay. The gradient-norm panel shows a stable baseline of

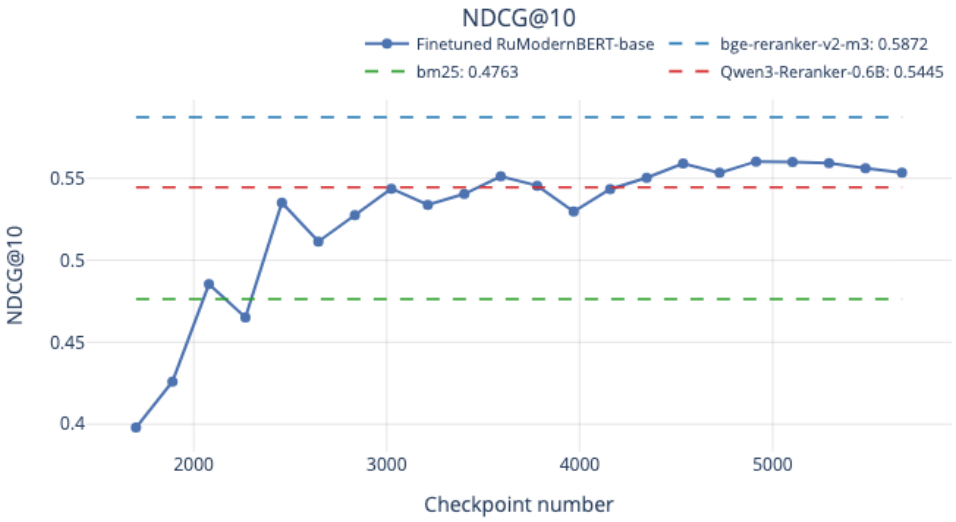


Fig. 5.2. NDCG@10 on the rusBEIR macro-average (9 datasets, out-of-distribution) across all evaluation checkpoints during fine-tuning of RuModernBERT-base (8HN+2RN). Dashed horizontal lines mark the baseline scores for BM25, Qwen3-Reranker-0.6B, and bge-reranker-v2-m3; exact values are in Table 5.1.

approximately 5–10 with isolated outliers reaching 40–55, concentrated in the early training phase but recurring sporadically throughout the run. These spikes are most plausibly attributable to false negatives in the contrastive batch: when a randomly sampled in-batch negative or a mined hard negative is in fact semantically equivalent to the query's positive passage, the MNRL objective receives a contradictory signal and back-propagates a large gradient. Because gradient clipping at a maximum norm of 1.0 is applied at every step, the post-clip update written to the optimiser is bounded regardless of the raw gradient magnitude, so individual outlier steps do not push the parameters far from the surrounding trajectory. The overall stability of the loss and the monotonic improvement of the out-of-distribution NDCG@10 curve in Figure 5.2 support this interpretation: the clipping mechanism absorbs the occasional contradictory signal without compromising convergence.

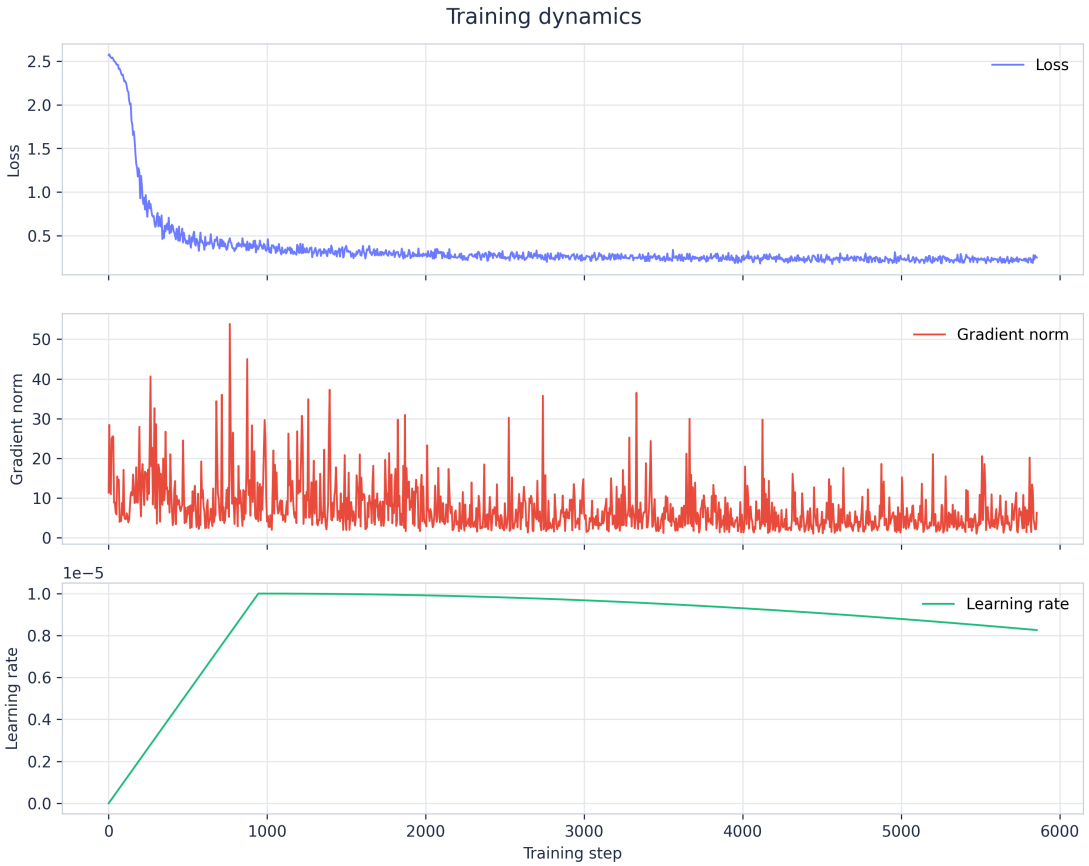


Fig. 5.3. In-training dynamics of the 8HN+2RN fine-tuning run of RuModernBERT-base: training loss (top), gradient norm before clipping (middle), and learning rate (bottom) versus optimisation step.

Our model vs. Qwen3-Reranker. RuModernBERT-base (8HN+2RN, checkpoint 4914) achieves NDCG@10 = 0.560 and MAP@10 = 0.432, outperforming Qwen3-Reranker-0.6B on all four metrics (NDCG +1.5 pp, MAP +2.2 pp, MRR +0.9 pp, P@3 +0.4 pp) despite having four times fewer parameters. This supports the hypothesis in O1 that Russian-specific pre-training compensates for reduced parameter count on Russian-centric benchmarks.

WikiFacts-Sents. One dataset where RuModernBERT-base leads all models on all metrics is wikifacts-sents (sentence-level fact retrieval over Russian Wikipedia), suggesting that Russian-native representations are particularly effective at fine-

grained factual matching in this domain.

5.2 Qualitative Error Analysis

To complement the quantitative results in Section 5.1, this section examines failure cases from the fine-tuned RuModernBERT-base (8HN+2RN) checkpoint. The goal is not only to record where relevant documents fall in the ranked list, but to identify what kind of mistake the cross-encoder makes when no relevant document reaches the top results.

5.2.1 Sampling Procedure

Failure cases are sampled from the nine active rusBEIR evaluation datasets used in Section 4.4.3. The two contaminated scientific datasets, ruscibench-retrieval_dev and rus-scidocs_test, are not part of this active set. A query counts as a *failure* when no relevant document appears in the top-5 reranked results. Under this criterion, rus-trec-covid contributed no sampleable instances: the dataset is small, and every query retained at least one positive document in the top-5 reranked results. The manual analysis therefore covers the remaining eight active datasets. Up to 20 failures per dataset were sampled with a fixed random seed (seed $= 42$), giving an initial pool of 160 cases. Cases in which the correct document was absent from the BM25 candidate set for that query were removed. In these cases, the cross-encoder never received the correct document as input, so the error belongs to the first-stage retriever rather than the reranker. The final analysis set contains **132 samples** across the eight datasets.

Each sample was manually read and assigned an error group label. The grouping criterion was the *nature of the cross-encoder's mistake*, not the query

topic or dataset domain, yielding seven distinct error types.

5.2.2 Error Taxonomy

Table 5.2 summarises the seven error groups and their frequencies.

TABLE 5.2
Cross-encoder error taxonomy. Frequencies are counts and percentages over the 132-sample analysis set (seed = 42, top-5 failure criterion, 8 rusBEIR datasets).

ID	Error type	Count	%
B	Same-topic instance disambiguation	50	38
D	Surface lexical trap	26	20
E	Answer granularity mismatch	22	17
C	Named entity confusion	10	8
G	Lay-to-technical bridge failure	10	8
A	Temporal / numerical specificity	7	5
F	Source / perspective constraint	7	5
	Total	**132**	**100**

B: Same-topic instance disambiguation (50 cases, 38%). The most frequent failure occurs when the corpus contains multiple documents covering the same subject and the cross-encoder selects the wrong specific instance. The model correctly identifies the relevant domain or entity but cannot distinguish between, for example, different tournament results for the same athlete, different news events of the same type, or different sentences drawn from the same Wikipedia article. From ria-news: the query asks about a motorcyclist hitting a guardrail in *south* Moscow; the top-ranked document describes a nearly identical accident in *northwest* Moscow. In the sentence-level retrieval datasets (XQuAD-Sentences, SberQuAD), this error accounts for nearly all failures: the model retrieves a sentence from the correct topic cluster but not the one that explicitly states the queried fact. This pattern

suggests that the cross-encoder struggles with fine-grained instance discrimination when documents are near-duplicates at the topic level.

D: Surface lexical trap (26 cases, 20%). The second most common error arises when high surface lexical overlap between the query and a retrieved document masks a semantic mismatch. The model follows shared keywords without checking whether the document actually satisfies the query intent. A clear example: the query asks about a *virtual* house burglary (theft of a Facebook game item), but the model retrieves a real-world home-burglary report, discarding the critical modifier. In nfcorpus, queries for "airport scanners" match papers on CT scanners in clinical settings, a different scanning technology in a different institutional context. A query about the month of the Chernobyl explosion retrieves a passage from a Ukrainian miniseries that mentions the April 1986 dates as narrative backdrop rather than factual record.

E: Answer granularity mismatch (22 cases, 17%). The model retrieves a topically correct document that does not surface the specific answering proposition the query requires. The cross-encoder assigns high relevance to broad narrative coverage when the query demands a direct factual statement. In RuBQ, the query "In which ocean is the Mariana Trench?" ranks a document describing the trench's V-shaped profile and exceptional depth above a document whose opening sentence reads "oceanic trench in the western Pacific Ocean." Likewise, "In what year was the Hermitage built?" retrieves a 2015 exhibition announcement at the Hermitage rather than the passage stating "arose in 1764 as Catherine II's private collection." These cases suggest that the cross-encoder over-weights topical relatedness and under-weights whether the document contains the specific answer.

C: Named entity confusion (10 cases, 8%). The model retrieves a document about an entity whose name partially or fully matches the query but which refers to a different referent. The query "What science fiction novel did Alexei Nikolaevich Tolstoy write?" ranks a passage about Lev Nikolaevich Tolstoy (a different writer sharing the surname) at the top. Similarly, "What was the name of the first human cosmonaut?" retrieves an article about Valentina Gagarina (Yuri Gagarin's widow) rather than the cosmonaut himself. In the news domain, a query specifying the French *premier* (Fillon) discussing missile defence in Moscow is answered with a passage about the French *president* (Hollande) discussing Syria, a title confusion within the same national administration.

G: Lay-to-technical bridge failure (10 cases, 8%). This failure type is specific to domain-specialised corpora (nfcorpus and rus-fiqa), where queries are written in informal lay language while the corpus consists of technical documents. The cross-encoder cannot reliably bridge between the general phrasing of the query and the appropriate technical document, often retrieving a paper that shares surface terminology but addresses an unrelated clinical or financial question. The query "soil health" retrieves a study on MRSA contamination near pig farms, which discusses soil in a veterinary-epidemiological context entirely unrelated to the agronomic intent of the query. "Tongue worm" retrieves a tapeworm (*Diphyllobothrium*) case report instead of the correct document on pentastomid parasites. The model was trained mostly on general Russian text, so it has little domain-specific signal for resolving terminological ambiguity in specialised corpora.

A: Temporal and numerical specificity failure (7 cases, 5%). The model retrieves a topically correct document that fails to satisfy a date, year, count, or other numerical constraint in the query. A query about Moscow bicycle-parking rankings

"in 2012" retrieves the 2011 edition of the same recurring rating; both documents are structurally and lexically near-identical. A query about "gold prices rising on Tuesday on eurozone fears" is satisfied by a similar article from a different Tuesday, indicating that the model treats recurring event templates as interchangeable. In nfcorpus, the query "antioxidant content in 300 foods" retrieves the widely cited "3,100 foods" study, numerically similar enough that the cross-encoder does not penalise the mismatch.

F: Source and perspective constraint failure (7 cases, 5%). This category arises when a query identifies a specific actor whose statement or action is sought, but the model retrieves a document describing the same event from a different actor's viewpoint. The query "Tymoshenko is tried for falsifying Cabinet directives, *said Azarov*" is answered with Tymoshenko's own courtroom statement rather than the prime minister's public accusation. A query specifying which Turkish referee would officiate the Chelsea-Spartak Champions League match retrieves match previews and tactical analyses, ignoring the source constraint. The query about which Soviet cosmonauts participated in the Apollo-Soyuz mission retrieves a passage about the American astronaut Vance Brand, reversing the national perspective.

5.2.3 Discussion

The error distribution is strongly skewed. Same-topic instance disambiguation (B) and surface lexical traps (D) together account for 58% of the analysed failures, while the remaining five types share the other 42% roughly evenly.

This pattern suggests that the cross-encoder is better at recognising topical overlap than at separating near-identical candidates. When several documents have almost the same surface form, such as the same entity, the same event type, or

the same recurring news template, the model often chooses a plausible neighbour instead of the exact answer. The difficult cases are usually decided by a small piece of evidence: a modifier such as “virtual” or “south”, a year such as “2012”, or a specific numeric value inside otherwise similar text.

The answer granularity mismatch errors (E, 17%) show a related problem. The model gives high scores to documents that discuss the right topic but do not directly state the answer, even when the query asks for a short factual response. This is consistent with the cross-encoder training objective, which optimises for binary or graded relevance labels without an explicit signal for answer presence or span coverage.

The lay-to-technical bridge failures (G) are concentrated entirely in nfcorpus and rus-fiqa, confirming that the model’s generalisation to specialised biomedical and financial terminology is limited. Changing the negative-sampling strategy alone is unlikely to remove these errors. They likely require domain-specific fine-tuning or domain-adapted pre-trained weights.

The qualitative analysis points to two useful directions for future work. First, mining hard negatives from temporally adjacent news articles or from same-topic Wikipedia passages would create contrastive pairs that explicitly require fine-grained instance discrimination, directly targeting the dominant error type. Second, augmenting the training signal with span-level answer-presence annotations could encourage the model to rank documents that directly contain the answering proposition above topically related but non-answering documents, addressing the granularity and source constraint failures.

5.3 Research Objectives Assessment

5.3.1 O1: Architecture Selection and Comparative Effectiveness

RuModernBERT-base satisfies the two criteria set out in O1: a deployment-practical parameter count (150 M, single-GPU inference) and a native 8,192-token context window. The comparative evaluation in Section 5.1 confirms the secondary criterion (competitive ranking quality relative to substantially larger multilingual cross-encoders) with RuModernBERT-base outperforming Qwen3-Reranker-0.6B (596 M) on all four reported metrics. **O1 is assessed as fully met.** Correspondingly, RQ1 receives a partial answer: the specialised Russian cross-encoder surpasses one of the two evaluated general-purpose multilingual competitors, while bge-reranker-v2-m3 remains the strongest baseline by a consistent margin across all datasets.

5.3.2 O2: Heterogeneous Training Corpus

The training corpus was constructed as specified, combining Russian-language general retrieval data with English domain-specific sources (NQ, SWIM-IR). However, the contribution of the English domain-specific component was not isolated in a controlled ablation: the corpus composition is held fixed across all experimental conditions in Chapter 4. Whether the English domain-specific data improves precision on Russian texts containing English vocabulary therefore remains unquantified. **O2 is assessed as partially met**: the corpus was successfully constructed, but the stated hypothesis was not empirically tested. This is noted as a limitation in Section 5.4.

5.3.3 O3: Ablation of Pipeline Optimisations

O3a: Consistency filtering. As reported in Section 4.5, consistency filtering at $k = 30$ yields no statistically meaningful improvement over unfiltered training (Δ NDCG@10 < 0.002), and filtering at $k = 5$ produces a measurable degradation due to over-filtering. The technique adds pipeline complexity without a measurable benefit at the current data scale. This null result is itself an empirically informative finding: it establishes that simple threshold-based consistency filtering does not reliably improve cross-encoder quality given the sampling budgets evaluated here. **O3a is assessed as answered.**

O3b: Hard negative sampling. The ablation in §4.6 establishes two findings. First, removing mined hard negatives entirely (0HN+2RN) reduces NDCG@10 by approximately ten percentage points relative to the mixed conditions, confirming that in-batch random negatives alone are insufficient for high-precision reranking. Second, increasing the per-query hard-negative count from 8 to 15 produces no measurable change on any reported metric, indicating that the discriminative signal saturates well before the mined pool is exhausted. The 8HN+2RN configuration is therefore adopted as the headline setting because it attains the same quality as 15HN+2RN at lower per-step compute and memory cost. **O3b is assessed as answered.**

5.4 Limitations and Future Work

5.4.1 Fixed Choice of the Retrieval Model

Throughout this work, a single dense retrieval model $\mathcal{E}$ — Qwen3-Embeddings-0.6b — was used for both consistency filtering and hard negative mining. This choice was not systematically validated against alternatives, and the effect of substituting a different retrieval model on downstream cross-encoder quality remains an open question.

A particularly interesting comparison would be between retrieval models of different capacity. A stronger model with a larger parameter count and broader training coverage produces a richer, more nuanced embedding space in which a wider variety of semantically related passages cluster near a given query. Under such a model, the margin-based hard negative criterion may admit a larger fraction of genuinely relevant passages as candidate negatives, increasing the risk of false-hard-negative contamination. Conversely, a weaker retrieval model — one with fewer parameters or trained on a narrower distribution — produces a coarser embedding space in which only the most prototypical, unambiguously relevant pairs are embedded close together. The consistency filtering criterion applied to such a space would therefore be more conservative, retaining only pairs whose relevance is robust and widely attested, potentially yielding a cleaner training signal at the cost of discarding more data.

This asymmetry motivates a systematic study of how retrieval model capacity interacts with filtering stringency and hard negative quality. Specifically, one could compare pipelines using the same cross-encoder architecture but with $\mathcal{E}$ varied across a range of model sizes, measuring the effect on both filtering attrition rates and final reranking quality. Such an experiment would provide principled guidance

for retrieval model selection in future training pipelines.

5.4.2 Absence of Dataset Sampling and Loss Weighting Strategies

The training pipeline samples batches from the multi-source corpus without applying dataset-level temperature sampling or weighted loss scaling. Temperature-based sampling controls the effective mixing ratio across datasets by raising per-dataset sampling probabilities to an inverse temperature, allowing over-represented sources to be down-weighted and smaller or higher-quality datasets to contribute more uniformly to gradient updates. Weighted loss scaling addresses a related but distinct problem: assigning different per-instance or per-dataset loss weights to reflect annotation quality or domain importance. Neither technique is evaluated in this work. As a result, the training signal is dominated by the largest datasets in the corpus, and the effect of dataset imbalance on cross-encoder quality — particularly on low-resource or domain-specific subsets — remains unquantified. Incorporating these strategies as additional ablation axes would be a natural extension of the present methodology.

5.4.3 SLERP Across Different-Seed Training Runs

The SLERP ablation in §4.7 merged checkpoints from runs that differ either in training data (V1, V2, V3) or in their position along a single trajectory. Neither setting produced a clear gain. A more principled setting that was not evaluated is merging the best checkpoints of multiple training runs with identical hyperparameters and data but different random seeds. Such runs would converge to comparable quality from independent initialisations and data orderings, so their

disagreements should be largely stochastic rather than data-driven — the regime in which checkpoint merging is most plausibly beneficial. This was not done because each additional run carries the same multi-GPU training cost as the experiments already reported, and the computational budget for this thesis did not allow training enough independent seeds to support the merge. A future extension would re-run the best configuration across several seeds and apply $\mathrm{multi_slerp}$ to the resulting checkpoints.

5.4.4 Absence of Optimiser Hyperparameter Tuning

The learning rate, weight decay, warm-up ratio, schedule shape, AdamW configuration, and MNRL temperature were fixed at conservative values rather than tuned, as noted in §4.3.1. Each full training run consumes the entire $8\times$H200 node for several hours, so a grid sweep across even a small subset of these axes was incompatible with also running the three filtering variants and four negative-composition conditions reported in Chapter 4. The compute budget was deliberately allocated to the data-side ablations hypothesised to dominate the outcome for a Russian-specialised cross-encoder; the optimiser settings were therefore inherited from common BERT fine-tuning practice without per-task validation. A more thorough study would re-run the best ablation configuration under a learning-rate scan in particular, since BERT fine-tuning is documented to be sensitive to this hyperparameter and its interaction with warm-up duration [72].

5.4.5 Limited Comparison of Cross-Encoder Base Models

This study fine-tunes a single cross-encoder base model, $\mathrm{RuModernBERT\text{-}base}$, and evaluates the effect of the negative sampling ratio. The comparison does

not cover several potentially relevant alternatives — in particular, models based on different pre-training corpora, tokenizers optimized for Russian morphology, or architectures with explicit multilingual alignment objectives. Extending the comparison to a broader set of base models would provide a more complete picture of the trade-offs between model size, language specialization, and reranking effectiveness in the Russian retrieval setting.

References

[1] C. Sharma, *Retrieval-augmented generation: A comprehensive survey of architectures, enhancements, and robustness frontiers*, 2025. arXiv: 2506.00054 [cs.IR]. [Online]. Available: https://arxiv.org/abs/2506.00054

[2] A. J. Oche, A. G. Folashade, T. Ghosal, and A. Biswas, *A systematic review of key retrieval-augmented generation (rag) systems: Progress, gaps, and future directions*, 2025. arXiv: 2507.18910 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2507.18910

[3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, *Bert: Pre-training of deep bidirectional transformers for language understanding*, 2019. arXiv: 1810.04805 [cs.CL]. [Online]. Available: https://arxiv.org/abs/1810.04805

[4] N. Reimers and I. Gurevych, *Sentence-bert: Sentence embeddings using siamese bert-networks*, 2019. arXiv: 1908.10084 [cs.CL]. [Online]. Available: https://arxiv.org/abs/1908.10084

[5] P. Lewis et al., *Retrieval-augmented generation for knowledge-intensive nlp tasks*, 2021. arXiv: 2005.11401 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2005.11401

[6] J. Lin, R. Nogueira, and A. Yates, *Pretrained transformers for text ranking: Bert and beyond*, 2021. arXiv: 2010.06467 [cs.IR]. [Online]. Available: https://arxiv.org/abs/2010.06467

[7] H. Déjean, S. Clinchant, and T. Formal, *A thorough comparison of cross-encoders and llms for reranking splade*, 2024. arXiv: 2403.10407 [cs.IR]. [Online]. Available: https://arxiv.org/abs/2403.10407

[8] R. Nogueira and K. Cho, *Passage re-ranking with bert*, 2020. arXiv: 1901.04085 [cs.IR]. [Online]. Available: https://arxiv.org/abs/1901.04085

[9] N. F. Liu et al., *Lost in the middle: How language models use long contexts*, 2023. arXiv: 2307.03172 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2307.03172

[10] G. Kovalev, M. Tikhomirov, E. Kozhevnikov, M. Kornilov, and N. Loukachevitch, *Building russian benchmark for evaluation of information retrieval models*, 2025. arXiv: 2504.12879 [cs.IR]. [Online]. Available: https://arxiv.org/abs/2504.12879

[11] P. Bajaj et al., *Ms marco: A human generated machine reading comprehension dataset*, 2018. arXiv: 1611.09268 [cs.CL]. [Online]. Available: https://arxiv.org/abs/1611.09268

[12] T. Blevins et al., *Breaking the curse of multilinguality with cross-lingual expert language models*, 2024. arXiv: 2401.10440 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2401.10440

[13] D. Zmitrovich et al., *A family of pretrained transformer language models for russian*, 2024. arXiv: 2309.10931 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2309.10931

[14] T. Kwiatkowski et al., "Natural questions: A benchmark for question answering research," *Transactions of the Association for Computational Linguistics*, vol. 7, L. Lee, M. Johnson, B. Roark, and A. Nenkova, Eds., pp. 453–466, 2019. doi: 10.1162/tacl_a_00276 [Online]. Available: https://aclanthology.org/Q19-1026/

[15] N. Thakur, J. Ni, G. H. Ábrego, J. Wieting, J. Lin, and D. Cer, *Leveraging llms for synthesizing training data across many languages in multilingual dense retrieval*, 2024. arXiv: 2311.05800 [cs.IR]. [Online]. Available: https://arxiv.org/abs/2311.05800

[16] S. E. Robertson, "The probability ranking principle in IR," *Journal of Documentation*, vol. 33, no. 4, pp. 294–304, 1977. doi: 10.1108/eb026647 [Online]. Available: https://doi.org/10.1108/eb026647

[17] K. Spärck Jones, "A statistical interpretation of term specificity and its application in retrieval," *Journal of Documentation*, vol. 28, no. 1, pp. 11–21, 1972, Reprinted in Journal of Documentation 60(5):493–502, 2004. doi: 10.1108/eb026526 [Online]. Available: https://doi.org/10.1108/eb026526

[18] S. E. Robertson and H. Zaragoza, "The probabilistic relevance framework: Bm25 and beyond," *Found. Trends Inf. Retr.*, vol. 3, pp. 333–389, 2009. [Online]. Available: https://api.semanticscholar.org/CorpusID:207178704

[19] M. Lu, C. Chen, and C. Eickhoff, *Pathway to relevance: How cross-encoders implement a semantic variant of bm25*, 2025. arXiv: 2502.04645 [cs.IR]. [Online]. Available: https://arxiv.org/abs/2502.04645

[20] T. Mikolov, K. Chen, G. Corrado, and J. Dean, *Efficient estimation of word representations in vector space*, 2013. arXiv: 1301.3781 [cs.CL]. [Online]. Available: https://arxiv.org/abs/1301.3781

[21] J. Pennington, R. Socher, and C. Manning, "GloVe: Global vectors for word representation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, A. Moschitti, B. Pang, and W. Daelemans, Eds., Doha, Qatar: Association for Computational Linguistics, Oct. 2014, pp. 1532–1543. doi: 10.3115/v1/D14-1162 [Online]. Available: https://aclanthology.org/D14-1162/

[22] J. Wang et al., *Utilizing bert for information retrieval: Survey, applications, resources, and challenges*, 2024. arXiv: 2403.00784 [cs.IR]. [Online]. Available: https://arxiv.org/abs/2403.00784

[23] A. Vaswani et al., *Attention is all you need*, 2023. arXiv: 1706.03762 [cs.CL]. [Online]. Available: https://arxiv.org/abs/1706.03762

[24] Y. Liu et al., *Roberta: A robustly optimized bert pretraining approach*, 2019. arXiv: 1907.11692 [cs.CL]. [Online]. Available: https://arxiv.org/abs/1907.11692

[25] L. Wang et al., *Text embeddings by weakly-supervised contrastive pre-training*, 2024. arXiv: 2212.03533 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2212.03533

[26] W. X. Zhao, J. Liu, R. Ren, and J.-R. Wen, *Dense text retrieval based on pretrained language models: A survey*, 2022. arXiv: 2211.14876 [cs.IR]. [Online]. Available: https://arxiv.org/abs/2211.14876

[27] N. Yadav, N. Monath, M. Zaheer, R. Fergus, and A. McCallum, *Adaptive retrieval and scalable indexing for k-nn search with cross-encoders*, 2024.

arXiv: 2405.03651 [cs.IR]. [Online]. Available: https://arxiv.org/abs/2405.03651

[28] A. V. Petrov, S. MacAvaney, and C. Macdonald, "Shallow cross-encoders for low-latency retrieval," in *Advances in Information Retrieval*. Springer Nature Switzerland, 2024, pp. 151–166, isbn: 9783031560637. doi: 10.1007/978-3-031-56063-7_10 [Online]. Available: http://dx.doi.org/10.1007/978-3-031-56063-7_10

[29] O. Khattab and M. Zaharia, "Colbert: Efficient and effective passage search via contextualized late interaction over bert," in *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, New York, NY, USA: ACM, 2020, pp. 39–48. doi: 10.1145/3397271.3401075 [Online]. Available: https://arxiv.org/abs/2004.12832

[30] R. Jha et al., *Jina-colbert-v2: A general-purpose multilingual late interaction retriever*, 2024. arXiv: 2408.16672 [cs.IR]. [Online]. Available: https://arxiv.org/abs/2408.16672

[31] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, *Beir: A heterogenous benchmark for zero-shot evaluation of information retrieval models*, 2021. arXiv: 2104.08663 [cs.IR]. [Online]. Available: https://arxiv.org/abs/2104.08663

[32] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei, *Multilingual e5 text embeddings: A technical report*, 2024. arXiv: 2402.05672 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2402.05672

[33] A. Conneau et al., "Unsupervised cross-lingual representation learning at scale," in *Proceedings of the 58th Annual Meeting of the Association*

for Computational Linguistics, D. Jurafsky, J. Chai, N. Schluter, and J. Tetreault, Eds., Online: Association for Computational Linguistics, Jul. 2020, pp. 8440–8451. doi: 10.18653/v1/2020.acl-main.747 [Online]. Available: https://aclanthology.org/2020.acl-main.747/

[34] X. Zhang et al., *Mgte: Generalized long-context text representation and reranking models for multilingual text retrieval*, 2024. arXiv: 2407.19669 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2407.19669

[35] Y. Zhang et al., *Qwen3 embedding: Advancing text embedding and reranking through foundation models*, 2025. arXiv: 2506.05176 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2506.05176

[36] B. Li, H. Zhou, J. He, M. Wang, Y. Yang, and L. Li, *On the sentence embeddings from pre-trained language models*, 2020. arXiv: 2011.05864 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2011.05864

[37] S. R. Bowman, G. Angeli, C. Potts, and C. D. Manning, *A large annotated corpus for learning natural language inference*, 2015. arXiv: 1508.05326 [cs.CL]. [Online]. Available: https://arxiv.org/abs/1508.05326

[38] A. Williams, N. Nangia, and S. R. Bowman, *A broad-coverage challenge corpus for sentence understanding through inference*, 2018. arXiv: 1704.05426 [cs.CL]. [Online]. Available: https://arxiv.org/abs/1704.05426

[39] D. Cer, M. Diab, E. Agirre, I. Lopez-Gazpio, and L. Specia, "Semeval-2017 task 1: Semantic textual similarity multilingual and crosslingual focused evaluation," in *Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017)*, Association for Computational Linguistics, 2017. doi: 10.18653/v1/s17-2001 [Online]. Available: http://dx.doi.org/10.18653/v1/S17-2001

[40] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, *M3-embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation*, 2025. arXiv: 2402.03216 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2402.03216

[41] S. Xiao, Z. Liu, Y. Shao, and Z. Cao, *Retromae: Pre-training retrieval-oriented language models via masked auto-encoder*, 2022. arXiv: 2205.12035 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2205.12035

[42] A. Yang et al., *Qwen3 technical report*, 2025. arXiv: 2505.09388 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2505.09388

[43] Z. Yang et al., *Hotpotqa: A dataset for diverse, explainable multi-hop question answering*, 2018. arXiv: 1809.09600 [cs.CL]. [Online]. Available: https://arxiv.org/abs/1809.09600

[44] C. Goddard et al., *Arcee's mergekit: A toolkit for merging large language models*, 2025. arXiv: 2403.13257 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2403.13257

[45] R. Nogueira, Z. Jiang, R. Pradeep, and J. Lin, "Document ranking with a pretrained sequence-to-sequence model," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, T. Cohn, Y. He, and Y. Liu, Eds., Online: Association for Computational Linguistics, Nov. 2020, pp. 708–718. doi: 10.18653/v1/2020.findings-emnlp.63 [Online]. Available: https://aclanthology.org/2020.findings-emnlp.63/

[46] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, *Roformer: Enhanced transformer with rotary position embedding*, 2023. arXiv: 2104.09864 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2104.09864

[47] L. Bonifacio et al., *Mmarco: A multilingual version of the ms marco passage ranking dataset*, 2022. arXiv: 2108.13897 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2108.13897

[48] J. Liu, Y. Ma, R. Zhao, J. Zheng, Q. Ma, and Y. Kang, *Listconranker: A contrastive text reranker with listwise encoding*, 2025. arXiv: 2501.07111 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2501.07111

[49] Y. Sun et al., *Circle loss: A unified perspective of pair similarity optimization*, 2020. arXiv: 2002.10857 [cs.CV]. [Online]. Available: https://arxiv.org/abs/2002.10857

[50] S. Xiao, Z. Liu, P. Zhang, N. Muennighoff, D. Lian, and J.-Y. Nie, *C-pack: Packed resources for general chinese embeddings*, 2024. arXiv: 2309.07597 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2309.07597

[51] X. Xie et al., *T2ranking: A large-scale chinese benchmark for passage ranking*, 2023. arXiv: 2304.03679 [cs.IR]. [Online]. Available: https://arxiv.org/abs/2304.03679

[52] B. Warner et al., *Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference*, 2024. arXiv: 2412.13663 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2412.13663

[53] DeepVK, *RuModernBERT-base model card*, https://huggingface.co/deepvk/RuModernBERT-base, Hugging Face model repository. Accessed: 2026-05-23, 2025.

[54] S. Gururangan et al., *Don't stop pretraining: Adapt language models to domains and tasks*, 2020. arXiv: 2004.10964 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2004.10964

[55] A. Snegirev, M. Tikhonova, A. Maksimova, A. Fenogenova, and A. Abramov, *The russian-focused embedders' exploration: Rumteb benchmark and russian embedding model design*, 2025. arXiv: 2408.12503 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2408.12503

[56] I. Rybin, V. Korablinov, P. Efimov, and P. Braslavski, "Rubq 2.0: An innovated russian question answering dataset," in *The Semantic Web: 18th International Conference, ESWC 2021, Virtual Event, June 6–10, 2021, Proceedings*, Berlin, Heidelberg: Springer-Verlag, 2021, pp. 532–547, isbn: 978-3-030-77384-7. doi: 10.1007/978-3-030-77385-4_32 [Online]. Available: https://doi.org/10.1007/978-3-030-77385-4_32

[57] P. Efimov, A. Chertok, L. Boytsov, and P. Braslavski, "Sberquad – russian reading comprehension dataset: Description and analysis," in *Experimental IR Meets Multilinguality, Multimodality, and Interaction*, A. Arampatzis et al., Eds., Cham: Springer International Publishing, 2020, pp. 3–15, isbn: 978-3-030-58219-7.

[58] X. Zhang et al., *Making a miracl: Multilingual information retrieval across a continuum of languages*, 2022. arXiv: 2210.09984 [cs.IR]. [Online]. Available: https://arxiv.org/abs/2210.09984

[59] Y. Qu et al., *Rocketqa: An optimized training approach to dense passage retrieval for open-domain question answering*, 2021. arXiv: 2010.08191 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2010.08191

[60] N. Banar, E. Lotfi, and W. Daelemans, *Beir-nl: Zero-shot information retrieval benchmark for the dutch language*, 2024. arXiv: 2412.08329 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2412.08329

[61] J. Zhan, J. Mao, Y. Liu, J. Guo, M. Zhang, and S. Ma, *Optimizing dense retrieval model training with hard negatives*, 2021. arXiv: 2104.08051 [cs.IR]. [Online]. Available: https://arxiv.org/abs/2104.08051

[62] C.-Y. Chuang, J. Robinson, L. Yen-Chen, A. Torralba, and S. Jegelka, *Debiased contrastive learning*, 2020. arXiv: 2007.00224 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2007.00224

[63] A. van den Oord, Y. Li, and O. Vinyals, *Representation learning with contrastive predictive coding*, 2019. arXiv: 1807.03748 [cs.LG]. [Online]. Available: https://arxiv.org/abs/1807.03748

[64] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, *A simple framework for contrastive learning of visual representations*, 2020. arXiv: 2002.05709 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2002.05709

[65] L. Gao, Y. Zhang, J. Han, and J. Callan, *Scaling deep contrastive learning batch size under memory limited setup*, 2021. arXiv: 2101.06983 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2101.06983

[66] F. Feng, Y. Yang, D. Cer, N. Arivazhagan, and W. Wang, *Language-agnostic bert sentence embedding*, 2022. arXiv: 2007.01852 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2007.01852

[67] Z. Huang, P. Yu, and J. Allan, *Cross-lingual knowledge transfer via distillation for multilingual information retrieval*, 2023. arXiv: 2302.13400 [cs.IR]. [Online]. Available: https://arxiv.org/abs/2302.13400

[68] C. Xu, S. Guan, D. Greene, and M.-T. Kechadi, *Benchmark data contamination of large language models: A survey*, 2024. arXiv: 2406.04244 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2406.04244

[69] C. Deng, Y. Zhao, X. Tang, M. Gerstein, and A. Cohan, *Investigating data contamination in modern benchmarks for large language models*, 2024. arXiv: 2311.09783 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2311.09783

[70] M. Eriksson et al., *Can we trust ai benchmarks? an interdisciplinary review of current issues in ai evaluation*, 2025. arXiv: 2502.06559 [cs.AI]. [Online]. Available: https://arxiv.org/abs/2502.06559

[71] J. Johnson, M. Douze, and H. Jégou, *Billion-scale similarity search with gpus*, 2017. arXiv: 1702.08734 [cs.CV]. [Online]. Available: https://arxiv.org/abs/1702.08734

[72] M. Mosbach, M. Andriushchenko, and D. Klakow, *On the stability of fine-tuning bert: Misconceptions, explanations, and strong baselines*, 2021. arXiv: 2006.04884 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2006.04884

[73] N. Craswell, B. Mitra, E. Yilmaz, and D. Campos, *Overview of the trec 2020 deep learning track*, 2021. arXiv: 2102.07662 [cs.IR]. [Online]. Available: https://arxiv.org/abs/2102.07662

[74] I. Loshchilov and F. Hutter, *Decoupled weight decay regularization*, 2019. arXiv: 1711.05101 [cs.LG]. [Online]. Available: https://arxiv.org/abs/1711.05101

[75] P. Micikevicius et al., *Mixed precision training*, 2018. arXiv: 1710.03740 [cs.AI]. [Online]. Available: https://arxiv.org/abs/1710.03740

[76] O. Sainz, J. A. Campos, I. García-Ferrero, J. Etxaniz, O. L. de Lacalle, and E. Agirre, *Nlp evaluation in trouble: On the need to measure llm data contamination for each benchmark*, 2023. arXiv: 2310.18018 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2310.18018

[77] M. Artetxe, S. Ruder, and D. Yogatama, *On the cross-lingual transferability of monolingual representations*, 2020. arXiv: 1910.11856 [cs.CL]. [Online]. Available: https://arxiv.org/abs/1910.11856

Statement of Individual Contributions

This thesis was written jointly by **Nikita Borisov** and **Dmitrii Dydalin**. The division of work is summarized below. Sections not listed were prepared jointly.

Methodology and implementation

- **Source dataset selection, schema unification, cleaning, and surrogate ID generation** (§3.2.1, §4.2): jointly by N. Borisov and D. Dydalin.
- **Train/validation splitting via bipartite-component algorithm** (§3.2.2): D. Dydalin.
- **Consistency filtering** — algorithm design (retrieval-rank and virtual-rank criterion), choice of retrieval model and thresholds, FAISS-based execution at scale, and ablation design (§3.2.3, §4.2.2, §4.5): N. Borisov.
- **Hard negative mining** — margin-based selection criterion, adaptive search depth, cross-lingual positive exclusion, and GPU-FAISS implementation on sm_90 (§3.2.4, §4.2.3, §4.6): D. Dydalin.
- **Cross-lingual augmentation and final corpus compilation** (§3.2.5): D. Dy-

dalin.

- **Architecture selection, dataset-aware batch sampler, and cross-rank schedule synchronization protocol** (§3.3.1, §3.3.2): N. Borisov.
- **Loss function design, negative composition, and gradient caching integration** (§3.3.3): D. Dydalin.
- **SLERP checkpoint merging — procedure and ablation** (§4.7): D. Dydalin.
- **Evaluation protocol** — benchmark selection, contamination analysis, two-stage pipeline setup, and baseline collection (§3.4, §4.4, §5.1): N. Borisov.

Writing

- **Chapter 2 (Literature Review), §§2.1–2.6**: N. Borisov.
- **Chapter 2 (Literature Review), §§2.7–2.9**: D. Dydalin.
- **Chapter 3 (Methodology)**: D. Dydalin.
- **Chapter 4 (Experiments)**: written jointly, with primary responsibility as follows: D. Dydalin — Software and Computational Environment (§4.1), Training Data (§4.2), Training Configuration (§4.3), Ablation: Hard Negative Sampling (§4.6), and SLERP Checkpoint Merging (§4.7); N. Borisov — Evaluation Protocol (§4.4) and Ablation: Consistency Filtering (§4.5).
- **Chapter 5 (Evaluation and Discussion)**: N. Borisov — Quantitative Evaluation (§5.1) and Qualitative Error Analysis (§5.2); D. Dydalin — Limitations and Future Work (§5.4).
- **Chapter 1 (Introduction)**: N. Borisov.

Both authors jointly reviewed, revised, and proofread the complete manuscript.

Appendix A

RusBEIR Evaluation Datasets: Contamination Risk Analysis

Table A.1 provides a complete per-dataset contamination analysis for all eleven candidate RusBEIR datasets considered in §4.4.3. For each dataset, the table identifies the passage corpus source, any training dataset sharing that source, the type of contamination (direct or corpus-level), and the assigned evaluation weight.

Wikipedia corpus overlap (swim_ir). The swim_ir training dataset [15] generates synthetic retrieval queries over Wikipedia paragraphs in multiple languages, including Russian. Three evaluation datasets — rubq, sberquad-retrieval, and wikifacts-sents — draw their passage corpora from Russian Wikipedia, creating corpus-level overlap with swim_ir.

The overlap is classified as corpus-level rather than pair-level because the queries in these evaluation datasets originate from an independent construction process: Wikidata knowledge-graph annotations (rubq), reading comprehension annotations (sberquad-retrieval), and factual probes created for the rusBEIR bench-

mark (wikifacts-sents). The cross-encoder has seen these passage texts paired with *different* synthetic queries during training, not with the evaluation queries. This reduces but does not eliminate the contamination concern.

English Wikipedia overlap (nq). rus-xquad and rus-xquad-sentences are Russian translations of XQuAD [77], which is derived from SQuAD v1.1 (240 English Wikipedia articles). The nq training dataset uses a broader set of English Wikipedia articles as its passage corpus. Overlap exists at the corpus level (shared Wikipedia source), but the specific 240 SQuAD articles are a small subset of NQ's coverage and the queries are independently derived.

Scientific corpus exclusions. ruscibench-retrieval is excluded because its passage corpus is the same ruSciBench collection used as a training source (ru_sci_bench). This constitutes direct passage-corpus contamination where the model may have been optimized on the same texts it is evaluated on. rus-scidocs shares the broader scientific literature domain with ru_sci_bench and is excluded as a precaution given the high corpus-family overlap risk.

TABLE A.1
Contamination risk analysis for all eleven RusBEIR candidate datasets. *Direct*: passage corpus is the same as a training source. *Corpus*: passages drawn from the same collection with independent queries/annotations. *Domain*: related domain but different corpus. *None*: no identified overlap.

Dataset	**Task**	**Passage source**	**Overlap. train set**	**Contam. type**	**Severity**	**Weight**
rubq	Open-domain QA	Russian Wikipedia	swim_ir	Corpus-level	Medium	$1/9$
ria-news	News IR	RIA Novosti	None	None	None	$1/9$
rus-nfcorpus	Bio-Medical IR	PubMed abstracts (transl.)	None	None	None	$1/9$
rus-trec-covid	Bio-Medical IR	CORD-19 (transl.)	None	None	None	$1/9$
sberquad-ret.	Reading comp. IR	Russian Wikipedia	swim_ir	Corpus-level	Medium	$1/9$
wikifacts-sents	Fact retrieval	Russian Wikipedia (sentences)	swim_ir	Corpus-level	Medium	$1/9$
rus-fiqa	Financial QA	StackExch. Financial (transl.)	ru_stackoverflow	Domain	Low	$1/9$
rus-xquad	QA	English Wikipedia $\rightarrow$ RU (transl.)	nq	Corpus-level	Low–medium	$1/9$
rus-xquad-sents	QA (sent.-level)	English Wikipedia $\rightarrow$ RU (transl.)	nq	Corpus-level	Low–medium	$1/9$
ruscibench-ret.	Scientific IR	ruSciBench abstracts	ru_sci_bench	**Direct**	Critical	0
rus-scidocs	Scientific IR	SciDocs (transl.)	ru_sci_bench	Corpus family	High	0

Appendix B

Per-Dataset Evaluation Results

Tables B.1–B.4 report per-dataset scores for all four models across the nine active rusBEIR datasets (macro-averages match Table 5.1). **Bold** marks the best score per row. Model abbreviations: **BM25** = BM25 first-stage retrieval; **Qwen3** = Qwen3-Reranker-0.6B; **BGE** = bge-reranker-v2-m3; **Ours** = RuModernBERT-base 8HN+2RN, checkpoint 4914.

TABLE B.1
Per-dataset NDCG@10. Best per row in **bold**.

Dataset	BM25	Qwen3	BGE	Ours
NFCorpus	0.263	0.317	**0.321**	0.309
TREC-COVID	0.519	0.708	**0.758**	0.678
SberQuAD	0.639	0.676	**0.689**	0.673
RuBQ	0.273	0.459	**0.520**	0.472
RIA-News	0.573	0.615	**0.761**	0.734
WikiFacts-Sents	0.099	0.131	0.143	**0.150**
FiQA	0.166	0.245	**0.301**	0.271
XQuAD	0.870	0.946	**0.962**	0.939
XQuAD-Sents	0.694	0.803	**0.831**	0.816
Macro-avg	0.455	0.545	**0.587**	0.560

TABLE B.2
Per-dataset MAP@10. Best per row in **bold**.

Dataset	BM25	Qwen3	BGE	Ours
NFCorpus	0.093	0.117	**0.118**	0.112
TREC-COVID	0.011	0.018	**0.019**	0.017
SberQuAD	0.546	0.581	**0.596**	0.576
RuBQ	0.213	0.393	**0.465**	0.411
RIA-News	0.534	0.562	**0.740**	0.709
WikiFacts-Sents	0.075	0.099	0.110	**0.119**
FiQA	0.128	0.193	**0.248**	0.217
XQuAD	0.849	0.939	**0.959**	0.929
XQuAD-Sents	0.660	0.786	**0.819**	0.800
Macro-avg	0.345	0.410	**0.453**	0.432

TABLE B.3
Per-dataset P@3. Best per row in **bold**.

Dataset	BM25	Qwen3	BGE	Ours
NFCorpus	0.308	0.361	**0.372**	0.353
TREC-COVID	0.600	0.840	**0.907**	0.807
SberQuAD	0.284	0.302	**0.312**	0.301
RuBQ	0.126	0.231	**0.269**	0.234
RIA-News	0.196	0.218	**0.264**	0.255
WikiFacts-Sents	0.055	0.073	0.082	**0.085**
FiQA	0.091	0.138	**0.181**	0.164
XQuAD	0.296	0.320	**0.323**	0.319
XQuAD-Sents	0.238	0.278	**0.284**	0.282
Macro-avg	0.244	0.307	**0.333**	0.311

TABLE B.4
Per-dataset MRR@10. Best per row in **bold**.

Dataset	BM25	Qwen3	BGE	Ours
NFCorpus	0.479	0.537	**0.571**	0.550
TREC-COVID	0.784	0.937	**0.960**	0.867
SberQuAD	0.429	**0.471**	0.469	0.455
RuBQ	0.287	0.490	**0.569**	0.505
RIA-News	0.534	0.643	**0.753**	0.709
WikiFacts-Sents	0.136	0.177	0.192	**0.207**
FiQA	0.205	0.308	**0.380**	0.351
XQuAD	0.849	0.942	**0.960**	0.929
XQuAD-Sents	0.660	0.789	**0.820**	0.802
Macro-avg	0.485	0.588	**0.630**	0.597